Deriving Hypercubes of Type Systems

Michael Stay
director.research@f1r3fly.io

L. Gregory Meredith
f1r3flyceo@gmail.com

Christian Wells
cb@holdea.net

Abstract

Barendregt's Lambda Cube organizes typed lambda calculi by enabling different dependencies between terms and types. We show how to derive an analogous hypercube of type systems from an arbitrary operational theory.

1 Introduction

Suppose we start with the untyped λ-calculus. How can we equip it with a type system? Barendregt [2] gave eight different type systems arranged in a cube. The type systems adjoin to the untyped calculus two special symbols $*$ and $\Box$ called "sorts" for tracking the difference between terms and types; they also adjoin a dependent product type code $\Pi_{x::A}B$. All the type systems share the same axioms and inference rules; they differ only in the choice of which sorts can be used in the formation and introduction rules for the dependent product.

Suppose instead we start with the asynchronous π-calculus [12, 7, 4]. Is there a similar way to equip it with a set of type systems? If so, is there a construction that can play a role similar to that of the dependent product? This paper answers in the affirmative: given any operational theory of the small-step semantics of a programming language, together with a finite declaration of the operational source sites to type, there is an algorithm we can use to construct a hypercube of type systems for that language that use type codes analogous to the dependent product.

2 Barendregt's lambda cube

Barendregt's lambda cube presents eight typed lambda calculi by varying which dependencies are allowed among terms and types. There are two sorts,

$$S = \{*, \Box\},$$

where $*$ is the sort of ordinary types and $\Box$ is the sort of kinds. The sort axiom common to all systems in the cube is

$$\frac{}{\vdash * : \Box}\ \textsc{Axiom}.$$

Let V be a set of variables. Raw terms are generated by

$$A, B, C, D \ ::= \ x \mid s \mid A\,B \mid \lambda x : A.\, B \mid \Pi x : A.\, B,$$

where $x \in V$ and $s \in S$. Contexts are generated by

$$\Gamma \ ::= \ \emptyset \mid \Gamma, x : A.$$

The one-step beta reduction relation is generated by the beta contraction rule

$$\frac{}{(\lambda x : A.\, B)\, C \to_\beta B[C/x]}\ \beta$$

and by compatibility with the term formers:

$$\frac{A \to_\beta A'}{A\, B \to_\beta A'\, B} \qquad \frac{B \to_\beta B'}{A\, B \to_\beta A\, B'} \qquad \frac{A \to_\beta A'}{\lambda x : A.\, B \to_\beta \lambda x : A'.\, B} \qquad \frac{B \to_\beta B'}{\lambda x : A.\, B \to_\beta \lambda x : A.\, B'}$$

$$\frac{A \to_\beta A'}{\Pi x : A.\, B \to_\beta \Pi x : A'.\, B} \qquad \frac{B \to_\beta B'}{\Pi x : A.\, B \to_\beta \Pi x : A.\, B'}\ .$$

Write $=_\beta$ for the beta-convertibility relation generated by $\to_\beta$.

The following typing rules are common to all eight systems:

$$\frac{\Gamma \vdash A : s}{\Gamma, x : A \vdash x : A}\ \textsc{Start} \qquad x \notin \Gamma \qquad\qquad \frac{\Gamma \vdash A : B \qquad \Gamma \vdash C : s}{\Gamma, x : C \vdash A : B}\ \textsc{Weakening} \qquad x \notin \Gamma$$

$$\frac{\Gamma \vdash C : \Pi x : A.\, B \qquad \Gamma \vdash D : A}{\Gamma \vdash C\, D : B[D/x]}\ \textsc{Application}$$

$$\frac{\Gamma \vdash A : B \qquad B =_\beta B' \qquad \Gamma \vdash B' : s}{\Gamma \vdash A : B'}\ \textsc{Conversion}.$$

The systems differ only in the pairs of sorts (s_1, s_2) allowed in the product and abstraction rules:

$$\frac{\Gamma \vdash A : s_1 \qquad \Gamma, x : A \vdash B : s_2}{\Gamma \vdash \Pi x : A.\, B : s_2}\ \textsc{Product}$$

$$\frac{\Gamma \vdash A : s_1 \qquad \Gamma, x : A \vdash B : s_2 \qquad \Gamma, x : A \vdash C : B}{\Gamma \vdash \lambda x : A.\, C : \Pi x : A.\, B}\ \textsc{Abstraction}.$$

The four possible pairs have the following meanings:

$$\begin{array}{ccl} (*, *) & : & \text{terms may depend on terms,} \\ (*, \Box) & : & \text{types may depend on terms,} \\ (\Box, *) & : & \text{terms may depend on types,} \\ (\Box, \Box) & : & \text{types may depend on types.} \end{array}$$

The eight vertices of the cube are obtained by choosing which of the three non-base pairs are present, always including the simply typed base pair $(*, *)$:

system	$(*, *)$	$(*, \Box)$	$(\Box, *)$	$(\Box, \Box)$
$\lambda \to$	×			
λP	×	×		
$\lambda 2$	×		×	
$\lambda \underline{\omega}$	×			×
$\lambda P2$	×	×	×	
$\lambda P \underline{\omega}$	×	×		×
$\lambda \omega$	×		×	×
λC	×	×	×	×

So $\lambda\!\rightarrow$ is the simply typed lambda calculus, λP adds dependent types, $\lambda 2$ adds polymorphism, $\lambda\underline{\omega}$ adds type operators, and λC (the calculus of constructions) contains all four product profiles.

3 Operational semantics

Universal algebra [13] is the study of algebraic structures, sets equipped with functions satisfying equations. Graphs, monoids, groups, rings, and modules over a fixed ring are a few examples of algebraic structures. Fields are not algebraic in this purely equational sense, because the usual definition involves the inequality requirement $0 \neq 1$ and a partial inverse operation. By contrast, vector spaces over a fixed field are algebraic: they are modules with scalar-multiplication operations indexed by that field.

In 1963, Lawvere [10] showed how an algebraic structure could be encoded into an "algebraic theory", roughly a category with finite products; that instances of the algebraic structure corresponded to product-preserving functors from the algebraic theory to the category of sets; and that homomorphisms of the algebraic structures correspond to natural transformations between the functors. This approach has proven very fruitful [1].

Algebraic theories can be extended in various ways: if the theory has all finite limits instead of just products, it's called an "essentially algebraic theory"; such theories let us equip sets with relations as well as functions. If it's also closed relative to the product, we have chosen to call it a "lambda theory"; lambda theories allow us to use variable binders in the syntax of our algebraic structures.

A "graph-structured" theory is one equipped with two distinguished objects V and E and with two morphisms $s, t : E \to V$. Models of such a theory contain directed multigraphs: algebraic structures with a set of vertices, a set of edges, and functions assigning a source and target vertex to each edge, satisfying no equations. Directed multigraphs may have parallel edges between the same pair of vertices as well as self loops.

3.1 Operational theories

A graph-structured lambda theory is an excellent toolbox for expressing the small-step operational semantics of a programming language or virtual machine. To emphasize this use case, we use the term "operational theory" and write Pr (for "process") instead of V and R (for "rewrite" or "reduction") instead of E. Processes (terms of the calculus, states of the virtual machine) get constructed by morphisms into Pr, while single-step state rewrites get constructed by morphisms into R, and each rewrite has a source and target process. From the closed structure, we get the ability to bind variables in our syntax and do capture-avoiding substitution in our rewrites.

Definition 3.1 (Finite presentation of an operational theory)**.** *A finite presentation $\mathcal{P}$ of an operational theory consists of:*

(i) *finitely many carrier objects other than* Pr *and* R*,*

(ii) *finitely many term constructor symbols between finite-limit and exponential expressions in those carriers other than $s, t : \mathrm{R} \to \mathrm{Pr}$,*

(iii) *finitely many relation symbols,*

(iv) *finitely many displayed equation schemas between compositions of the term constructors,*

(v) *and finitely many primitive rewrite constructors*

$$\Gamma_i, \Delta_i \vdash \rho_i : L_i \rightsquigarrow R_i.$$

The displayed source term L_i and target term R_i are part of the presentation. Subterm occurrences of L_i are presentation data, not invariant structure of an abstract quotient theory.

We keep three layers of presentation data separate. The display layer *consists of the raw syntax trees of the constructors, equations, and primitive rewrite sources and targets. This is the layer from which source occurrences are named. The* equational layer *consists of the listed equations; it generates the ordinary equality or structural congruence of processes in models of the operational theory. The* profile-matching layer *consists of finite relations between displayed type-bearing occurrences of equations, used only by the profile algorithm below. A displayed equation schema may include such a finite list of matched type-bearing occurrence pairs. Absent such a list, the algorithm uses the syntactic default matching defined in Section 4.3.*

This presentation-dependence is intentional. We will be constructing a set of modal operators from the displayed sources of rewrite rules by identifying specified source occurrences. This requires choosing a specific representative of an equivalence class for each source and choosing which displayed occurrences are operational sites. Structural equations may identify two displayed processes later, but they do not by themselves rename sites, add sites, or identify profile slots.

3.2 The untyped λ-calculus

The trivial operational theory is a theory of the simply-typed λ-calculus with the single generating type Pr. It automatically has lambda terms as part of the syntax, and beta reduction is an equality. However, it does not have any rewrites, so it does not give us a place to stand to reason about the small-step operational semantics of λ-calculus.

To model the untyped lambda calculus, we "wrap" the built-in syntax with generating term constructors. Notice below that there is no term constructor for variables, let alone a definition of free and bound variables or capture-avoiding substitution; this is all taken care of by the built-in binder and evaluation syntax. This wrapping layer gives us the ability to be much more specific about computational aspects: we explicitly track β reduction as a rewrite step and choose to use normal-order reduction (only allow reductions in the head position of an application) for our evaluation strategy.

Here's the presentation of the theory:

- Carriers other than Pr and R: None.
- Term constructors other than s, t :

$$\begin{aligned} p, q : \mathrm{Pr} &\vdash \mathrm{app}(p, q) : \mathrm{Pr} && \text{application} \\ k : \mathrm{Pr} \to \mathrm{Pr} &\vdash \mathrm{lam}(k) : \mathrm{Pr} && \text{abstraction} \end{aligned}$$

- Rewrite rules:

$$\begin{aligned} p : \mathrm{Pr}, k : \mathrm{Pr} \to \mathrm{Pr} &\vdash \mathrm{beta} : \mathrm{app}(\mathrm{lam}(k), p) \rightsquigarrow k(p) && \beta \\ q : \mathrm{Pr}, r : \mathrm{R} &\vdash \mathrm{head} : \mathrm{app}(s(r), q) \rightsquigarrow \mathrm{app}(t(r), q) && \text{head} \end{aligned}$$

Barendregt essentially generated from this theory a new one with sorts and a dependent product type former, equipped with inference rules for inductively defining the typing relation. We would like to come up with an algorithm that does most of that work for any operational theory, not just for λ-calculus.

3.3 Asynchronous π-calculus

As another example of a nontrivial operational theory, we present the asynchronous π-calculus. This is a simple model of concurrent execution via message passing; in it, one can represent race conditions and deadlocks. In general, a term may reduce in many inequivalent ways: in a race, it matters who wins.

- Carriers other than Pr and R:

$$\mathrm{Nm} \qquad \text{names}$$

- Term constructors other than s, t :

$$\begin{aligned} \vdash \mathrm{nil} : \mathrm{Pr} &\qquad \text{do-nothing} \\ p, q : \mathrm{Pr} \vdash p \mid q : \mathrm{Pr} &\qquad \text{parallel composition} \\ n, m : \mathrm{Nm} \vdash n!m : \mathrm{Pr} &\qquad \text{send} \\ n : \mathrm{Nm}, p : \mathrm{Nm} \to \mathrm{Pr} \vdash n?p : \mathrm{Pr} &\qquad \text{receive} \\ p : \mathrm{Pr} \vdash !p : \mathrm{Pr} &\qquad \text{replication} \\ p : \mathrm{Nm} \to \mathrm{Pr} \vdash \nu.p : \mathrm{Pr} &\qquad \text{restriction} \end{aligned}$$

- Equations:

$$\begin{aligned} p : \mathrm{Pr} \vdash p \mid \mathrm{nil} = p &\qquad \text{unit} \\ p, q : \mathrm{Pr} \vdash p \mid q = q \mid p &\qquad \text{commutativity} \\ p, q, r : \mathrm{Pr} \vdash (p \mid q) \mid r = p \mid (q \mid r) &\qquad \text{associativity} \\ \vdash \nu.\lambda x.\mathrm{nil} = \mathrm{nil} &\qquad \text{new nil} \\ p : \mathrm{Pr} \vdash !p = p \mid !p &\qquad \text{spawn} \\ p : \mathrm{Nm} \to \mathrm{Nm} \to \mathrm{Pr} \vdash \nu.\nu.p = \nu.\nu.\lambda x.\lambda y.p(x)(y) &\qquad \text{swap} \\ p : \mathrm{Nm} \to \mathrm{Pr}, q : \mathrm{Pr} \vdash (\nu.p) \mid q = \nu.\lambda x.(p(x) \mid q) &\qquad \text{scope extrusion} \end{aligned}$$

- Rewrite constructors:

$$\begin{aligned} q : \mathrm{Pr}, r : \mathrm{R} \vdash \mathrm{par}_1 : s(r) \mid q \rightsquigarrow t(r) \mid q &\qquad \mathrm{par}_1 \\ r_1, r_2 : \mathrm{R} \vdash \mathrm{par}_2 : s(r_1) \mid s(r_2) \rightsquigarrow t(r_1) \mid t(r_2) &\qquad \mathrm{par}_2 \\ n, m : \mathrm{Nm}, p : \mathrm{Nm} \to \mathrm{Pr} \vdash \mathrm{comm} : n!m \mid n?p \rightsquigarrow p(m) &\qquad \mathrm{comm} \\ r : \mathrm{Nm} \to \mathrm{R} \vdash \mathrm{ctx} : \nu.\lambda x.s(r(x)) \rightsquigarrow \nu.\lambda x.t(r(x)) &\qquad \mathrm{ctx} \end{aligned}$$

Some notes:

- We use Milner's original factorization of receive into a synchronization on a name followed by a continuation. The target of the comm rule evaluates the continuation p at the communicated name m. Similarly, ν is an operation on continuations; the displayed lambda notation belongs to the specified closed structure of the presentation.

- As an example of a race, consider the term

$$\nu.\lambda prize.\nu.\lambda finish.\nu.\lambda Alice.\nu.\lambda Bob.\\ (finish!Alice \mid finish!Bob \mid finish?\lambda winner.winner!prize).$$

  The two processes $finish!Alice$ and $finish!Bob$ race to deliver their message to the waiting continuation. The process above reduces to either

$$\nu.\lambda prize.\nu.\lambda finish.\nu.\lambda Alice.\nu.\lambda Bob.(finish!Bob \mid Alice!prize)$$

  or

$$\nu.\lambda prize.\nu.\lambda finish.\nu.\lambda Alice.\nu.\lambda Bob.(finish!Alice \mid Bob!prize).$$

  These are clearly two very different outcomes.

- The displayed source of the communication constructor is the ordered tree $n!m \mid n?p$. Even though commutativity later identifies this process with $n?p \mid n!m$, the display layer remembers the output occurrence as the left child and the input occurrence as the right child. The reversed representative creates no additional site unless it is separately listed as a rewrite constructor or site declaration.

- In the scope extrusion equation and the ctx rule, the fact that x does not appear to the left of the turnstile means that x is fresh relative to p, q, and r.

- If the π-calculus used only the par_1 rule, it would have interleaving semantics; par_2 is also necessary for true parallel execution.

We would like to generate from this theory a new one with sorts and a generalization of the dependent product, equipped with inference rules analogous to those in the definition of the lambda cube.

4 Dependently-typed operational theories

4.1 Generalizing the dependent product

The dependent product $\Pi_{x::A}B$ can be thought of as denoting "those processes that when applied to a term x of type A form a term of type B". In an operational theory, we would write it as $\mathsf{\Pi}(A, \lambda x.B)$, a morphism with two inputs: the first for the type of the dependent variable, and the second in which x is bound and may appear in the type B.

The first thing to notice when looking for an analogous dependent type for an arbitrary operational theory is the word "applied". Application is a feature of the λ-calculus; it does not appear in the π-calculus (nor in most other calculi). Instead, π-calculus uses a commutative juxtaposition operator. So the first generalization we have to make is to associate a dependent type to each displayed context K in which a process could be placed to cause a reduction to occur. A *candidate site* is a displayed subterm occurrence in the chosen source representative of a rewrite rule that we can replace with a hole to form a term context; in this paper, candidate sites have carrier Pr. A *declared site* is one of the candidate occurrences that the presentation explicitly selects for modal type formation. For example, in λ-calculus, the product-like completion declares the function-position occurrence, giving contexts like $K = \mathsf{app}([-], x)$, while in π-calculus, the receive completion declares the input occurrence in the displayed source $n!m \mid n?p$, giving the

displayed context $K = n!m \mid [-]$. The commutativity equation also makes $[-] \mid n!m$ extensionally equivalent after plugging a process into the hole, but it does not create a second modal site or identify it with the displayed one.

Secondly, the word "form" in that characterization refers to the subject reduction theorem, the fact that in the type systems of the lambda cube, reduction is type-preserving. Requiring a subject reduction theorem of all calculi is too onerous. For example, there are very few interesting properties of a π-calculus process that are invariant across all possible reductions; we are far more interested in things like "on which names can this process send?", and this set changes as the process evolves. Therefore, we would like to generalize to a modal "possibility" type, something like "those processes that when applied to a term x of type A may reduce to a term of type B in one step". For the λ-calculus, this definition and the one above are equivalent, but for the π-calculus, "those processes that when juxtaposed with a process x of type A may reduce to a term of type B in one step" is very different from "those processes that when juxtaposed with a process x of type A form a term of type B".

Traditionally, the modal type "possibly evolves to type X in one step" is denoted $\Diamond X$. Taken together with the notation for the dependent product that includes the variable on which the type depends, these two generalizations lead us to a notation like $\langle K \rangle_{x::A} B$, where we've placed the context K inside the diamond. This dependent type should denote "those processes that when placed in the context K with a process x of type A may reduce to a process of type B in one step". If the subterm at a site is placed into the resulting context, it may reduce to the target of the rewrite rule, so if we factor the source of a rewrite as $K[t, x]$, judge x to be of type A and the target of the rewrite to be of type B, we can say both $t :: \langle K \rangle_{x::A} B$ and $K[t, x] : \Diamond B$.

4.2 Typed contexts

An operational theory axiom has one variable context and one relation context. Dependent typing requires rules whose premises and conclusions live in different carrier-and-evidence contexts. For example, product formation has the form

$$\frac{\Gamma \mid \Delta \vdash A :: u \qquad \Gamma, x : \mathrm{Pr} \mid \Delta, e_x : x :: A \vdash B :: v}{\Gamma \mid \Delta \vdash \Pi(A, \lambda x.B) :: v} \ \Pi\text{-FORM}.$$

The second premise lives in the extended context $\Gamma, x : \mathrm{Pr} \mid \Delta, e_x : x :: A$, while the conclusion lives in $\Gamma \mid \Delta$. This is not a same-context axiom; it is an inference rule that discharges a carrier variable together with explicit typing evidence for that variable.

Accordingly, the construction below does not return a mere operational theory. It returns a finite presentation of a *dependently-typed operational theory*, a presentation of an operational theory equipped with typing judgments and specified inference rules.

4.3 Operational sites and local cubes

The examples above already contain the whole idea. A rewrite rule does not only say that one process may step to another; its displayed source also shows where an interaction can happen. The completion turns a chosen occurrence in that source into a modal dependent type former.

For beta reduction, the displayed source is

$$\mathsf{app}(\mathsf{lam}(t), a) \rightsquigarrow t(a).$$

If we choose the function-position occurrence $U = \mathsf{lam}(t)$, the surrounding context is

$$K[-] = \mathsf{app}([-], a).$$

The generated type says that a process behaves like a function when, after an argument is supplied and the process is placed in this displayed application context, one beta step can reach the target type. This is the application-site origin of the dependent product.

For communication in the asynchronous π-calculus, the displayed source is

$$n!m \mid n?k \rightsquigarrow k(m).$$

If we choose the input occurrence $U = n?k$, the surrounding context is

$$K[-] = n!m \mid [-].$$

The resulting receive type says that an input process, when placed next to an output supplying a message m along channel n, may communicate once and reach the continuation type. The continuation parameter k is not supplied by the context; it remains an ambient parameter of the rule.

These two examples also show why sites cannot simply be "all process subterms." The lambda-cube comparison wants the application/operator site, not the argument variable or the whole beta redex. The receive example may choose the input site without also choosing the output site. Thus site selection is part of the input data.

Definition 4.1 (Declared sites)**.** *A site-marked operational presentation consists of a finite operational presentation $\mathcal{P}$ together with a finite site specification $\mathfrak{S}$. For each primitive rewrite constructor*

$$\Gamma_i, \Delta_i \vdash \rho_i : L_i \rightsquigarrow R_i,$$

the set $\mathfrak{S}_i$ consists of selected occurrence addresses in the displayed *source tree L_i whose subterms have carrier* Pr*. For a site $s = (i, o)$, write U_s for the selected subterm and $K_s[-]$ for the displayed one-hole context satisfying*

$$L_i = K_s[U_s]$$

as raw displayed syntax.

Structural equations are deliberately kept separate from these addresses. An equation such as associativity or commutativity of $\mid$ may identify terms in models, but it does not move holes, create new sites, or merge declared sites. Likewise, it does not identify sort-profile slots unless the presentation also contains explicit profile-matching data. The fully formal version of this separation, including the default site policy and the finite matching relation, is given in Appendix A.

A site has three kinds of variables. The variables shared by the selected subterm and the context are the *index variables*

$$\vec{x}_s = \mathsf{FV}(K_s) \cap \mathsf{FV}(U_s).$$

The variables supplied only by the surrounding context are the *rely variables*

$$\vec{y}_s = \mathsf{FV}(K_s) \setminus \mathsf{FV}(U_s).$$

The variables needed by the selected subterm or the target, but not supplied by the one-hole context, are the *ambient variables*

$$\vec{z}_s = (\mathsf{FV}(U_s) \setminus \mathsf{FV}(K_s)) \cup (\mathsf{FV}(R_i) \setminus \mathsf{FV}(L_i)).$$

Their declarations form an ambient telescope Θ_s. Ambient variables are ordinary parameters; they are not axes of the hypercube.

The type former associated to a site is written

$$\langle K_s \rangle^{\vec{x}::\vec{A}}_{\vec{y}::\vec{B}} C.$$

This displayed type notation records the typed interface, not a context entry. The corresponding context always separates carriers from evidence. If $\vec{\aleph}_s$ and $\vec{\beth}_s$ are the carriers of the index and rely variables, then the carrier-and-evidence telescope is index-first:

$$\Gamma, \vec{x} : \vec{\aleph}_s, \vec{y} : \vec{\beth}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A}, \vec{e}_y : \vec{y} :: \vec{B}.$$

It should be read as follows: after the index data $\vec{x}$, together with index evidence $\vec{e}_x$, are fixed, and after the rely data $\vec{y}$, together with rely evidence $\vec{e}_y$, are supplied, placing the process in the displayed context $K_s[-]$ may take one step to a process of type C. The rely types may depend on the index carrier variables.

A declared site records the context for the site interaction itself. It does not, by itself, say that arbitrary rewrites can be propagated through that context. When the operational presentation explicitly supplies a contextual-closure constructor

$$r : P \rightsquigarrow Q \quad \longmapsto \quad \kappa_s(r) : K_s[P] \rightsquigarrow K_s[Q],$$

we write

$$\kappa_s \Vdash K_s[-]$$

and say that the site has *closure support*. This notation is a side condition on the operational presentation, not a typing judgment and not a consequence of structural congruence. The full formal definition, including ambient, index, and rely parameters, is given in Definition A.2.

Congruence transport for rewrites. Closure support is stable under equality of source and target terms. Thus, if $r : P \rightsquigarrow Q$, and the equational part of the presentation gives

$$P = P' \qquad \text{and} \qquad Q = Q',$$

then the same rewrite evidence may be transported to evidence

$$r' : P' \rightsquigarrow Q'.$$

Equivalently, rewrite evidence is interpreted modulo the structural equations of the presentation. This transport principle is separate from site extraction: structural equations may transport already existing rewrite evidence, but they do not create new displayed site addresses or force new profile-slot identifications.

The local cube for a site records which sorts are permitted for the index types, rely types, and target type. Its raw slots are

$$I_s = \{i_x \mid x \in \vec{x}_s\} \amalg \{r_y \mid y \in \vec{y}_s\} \amalg \{\tau_s\}.$$

A raw profile assigns to each slot either $*$ or $\square$, with the carrier superscript understood. We write profiles as $(\vec{\alpha}, \vec{\beta}; \gamma)$, where $\vec{\alpha}$ records the sorts of the index variables and $\vec{\beta}$ records the sorts of the rely variables. The displayed matching data of the presentation then generates a finite equivalence relation on slots; a quotient profile is a raw profile constant on those equivalence classes. A signature choice Σ_s is any chosen subset of the resulting quotient profile space including the all-$*$ element. The complete finite graph algorithm for this quotient is in Appendix A.

Example 4.2 (A nontrivial quotient)**.** *Suppose a site has three raw slots p, q, r, but the displayed matching data forces $p = q$. The raw profile cube has $2^3 = 8$ vertices. The quotient profile space has only the four profiles satisfying $p = q$:*

$$(*, *; *), \quad (*, *; \square), \quad (\square, \square; *), \quad (\square, \square; \square).$$

Thus quotienting is not a semantic closure over all provable equations; it is a finite, displayed, presentation-level operation.

5 The hypercube completion

We can now summarize the construction without the bookkeeping. An input object is a triple

$$A = (\mathcal{P}, \mathfrak{S}, \Sigma),$$

where $\mathcal{P}$ is a finite operational presentation, $\mathfrak{S}$ is a finite declaration of sites, and Σ chooses allowed quotient profiles for those sites. The completion adjoins sorts, proof-relevant typing evidence objects, Church-style annotations for binding constructors, the one-step possibility type $\Diamond$, and the contextual modalities selected by Σ.

Theorem 5.1 (Hypercube completion)**.** *For every finite site-and-signature presentation $A = (\mathcal{P}, \mathfrak{S}, \Sigma)$, there is a generated dependently typed operational presentation*

$$\mathcal{H}(A) = \mathcal{H}(\mathcal{P}, \mathfrak{S}, \Sigma).$$

Its generated type formers are exactly the one-step possibility type and the contextual modalities associated to the declared sites and chosen quotient profiles.

Construction sketch. For each relevant carrier X, adjoin the sort constants $*^X, \square^X$ and the proof-relevant typing display $p_X : \mathsf{Ev}_X \to X \times X$. For the identity-hole context, adjoin $\Diamond C$. For each site s and each profile $\omega \in \Sigma_s$, adjoin the contextual type former

$$\langle K_s \rangle^{\vec{x}::\vec{A}}_{\vec{y}::\vec{B}} C$$

with the profile-determined formation, introduction, elimination, and source-subterm rules. Binding constructors are refined by Church-style annotations. Delayed elimination through a context is an admissible principle, available only when the presentation supplies explicit closure support for that context. The formal list of generated symbols and rules is given in Appendices B and A; the main rules are displayed in the next sections. □

Proposition 5.2 (Finiteness)**.** *If $\mathcal{P}$, $\mathfrak{S}$, and Σ are finite, then $\mathcal{H}(\mathcal{P}, \mathfrak{S}, \Sigma)$ is finitely generated.*

Proof sketch. There are finitely many primitive rewrite constructors and finitely many declared site occurrences. Each source tree has finitely many free variables, so each site has finitely many index, rely, and ambient variables. The raw profile cube for a site is finite, and the consistency quotient is computed from a finite graph. Since Σ_s is finite for each site, only finitely many modal constructors and rule schemas are added. □

6 Typing and structural rules

For each relevant carrier object X, the generated presentation contains a proof-relevant typing display

$$p_X : \mathsf{Ev}_X \longrightarrow X \times X, \qquad p_X = \langle \mathsf{tm}_X, \mathsf{tp}_X \rangle.$$

The map p_X is not assumed to be monic. Its fiber over (a, b) is the object of evidence that a has classifier b. A typing-evidence assumption is therefore written with a name:

$$e : a ::^X b.$$

When the carrier is clear, the superscript is suppressed. A carrier declaration and its typing evidence are kept distinct. Thus a context containing a typed variable is written

$$\Gamma, x : X \mid \Delta, e_x : x ::^X A.$$

In conclusions and premises of inference rules we may still write $\Gamma \mid \Delta \vdash a ::^X A$ when the name of the evidence produced by the derivation is immaterial. In the context itself, however, the evidence assumption is explicit.

For every relevant carrier X, the sort rule is

$$\frac{}{\Gamma \mid \Delta \vdash *^X ::^X \square^X} \text{ SORT.}$$

The metavariable J. In the structural rules below, J is a metavariable for an atomic judgment of the generated presentation. It is not a new object-language judgment former. It ranges over the two judgmental forms of an operational theory: term judgments and relation judgments, with equations treated as a distinguished relation judgment. Thus, metasyntactically,

$$J \ ::= \ t : X \ \mid \ \phi(t_1, \ldots, t_n) \ \mid \ t = t',$$

where X ranges over carrier objects and ϕ ranges over relation symbols. Typing judgments $M :: A$ are displayed evidence judgments associated to the generated maps $p_X : \mathsf{Ev}_X \to X \times X$. For example, $M :: A$, $p = q$, and $r : P \rightsquigarrow Q$ are instances of J. The rewrite judgment $r : P \rightsquigarrow Q$ is still only syntactic sugar for the three atomic judgments

$$r : \mathrm{R}, \qquad s(r) = P, \qquad t(r) = Q.$$

Weakening and substitution act on J by acting on every term appearing in the judgment. In particular, $J[a/x]$ denotes the result of capture-avoiding substitution of a for x throughout the atomic judgment J.

The structured presentation has ordinary structural behavior for typed carrier-and-evidence contexts.

- Variable:

$$\frac{\Gamma \mid \Delta \vdash A ::^X u}{\Gamma, x : X \mid \Delta, e_x : x ::^X A \vdash x ::^X A} \text{ VAR.}$$

- Weakening:

$$\frac{\Gamma \mid \Delta \vdash J \qquad \Gamma \mid \Delta \vdash A ::^X u}{\Gamma, x : X \mid \Delta, e_x : x ::^X A \vdash J} \text{ WEAK.}$$

- Substitution:

$$\frac{\Gamma, x : X \mid \Delta, e_x : x ::^X A \vdash J \qquad \Gamma \mid \Delta \vdash a ::^X A}{\Gamma \mid \Delta \vdash J[a/x]}\ \text{SUB.}$$

7 Single-step rewriting and possibility

A rewrite judgment has the form

$$r : P \rightsquigarrow Q.$$

No reflexive-transitive closure is added by the completion. If a calculus wants reflexive, transitive, or symmetric rewrite witnesses, they must be supplied explicitly in the input presentation as primitive constructors or generated equations. Likewise, structural equations are not treated as rewrite witnesses and do not synthesize additional contextual sites. They remain equations, used by the ordinary conversion principle below.

The identity context $[-]$ gives the one-step possibility type

$$\Diamond C := \langle [-] \rangle\, C.$$

This is generated separately from the declared contextual sites. If the whole source occurrence of a primitive rewrite is explicitly declared as a site, it produces the same identity-hole context, but the default site policy excludes whole-source occurrences to avoid duplicating $\Diamond$.

Formation:

$$\frac{\Gamma \mid \Delta \vdash C :: \gamma}{\Gamma \mid \Delta \vdash \Diamond C :: \gamma}\ \Diamond\text{-FORM.}$$

Introduction:

$$\frac{\Gamma \mid \Delta \vdash r : P \rightsquigarrow Q \qquad \Gamma \mid \Delta \vdash Q :: C}{\Gamma \mid \Delta \vdash P :: \Diamond C}\ \Diamond\text{-INTRO}$$

The expression $P :: \Diamond C$ means that P can take one rewrite step to something of type C. There is no automatic return rule $P :: C \Rightarrow P :: \Diamond C$, unless reflexive rewrite instances are supplied in the input presentation.

Elimination:

$$\frac{\Gamma \mid \Delta \vdash P :: \Diamond C \qquad \Gamma, Q : \mathrm{Pr} \mid \Delta, r : P \rightsquigarrow Q, e_C : Q :: C \vdash J}{\Gamma \mid \Delta \vdash J}\ \Diamond\text{-ELIM}$$

The eliminator for $\Diamond$ is the existential eliminator for one-step reachability. From $P :: \Diamond C$, one may reason under fresh data

$$Q : \mathrm{Pr}, \qquad r : P \rightsquigarrow Q, \qquad e_C : Q :: C.$$

The conclusion must not depend on the chosen reduct or rewrite witness. Thus the rule uses some witness supplied by the $\Diamond$-evidence, but does not require the derivation of $P :: \Diamond C$ to have been obtained by a particular displayed $\Diamond$-introduction step.

8 Context-possibility types

Let $s \in \mathsf{Sites}(\mathcal{P})$ be a site. Write

$$L_i = K_s[U_s]$$

for the corresponding source factorization. Let

$$\vec{x} = \vec{x}_s, \qquad \vec{y} = \vec{y}_s, \qquad \vec{z} = \vec{z}_s.$$

Write Θ_s for the corresponding ambient telescope. The generic modal formation, introduction, and elimination rules are stable under arbitrary outer contexts; any ambient parameters needed by P, Q, or C are simply part of Γ. The site-specific source-subterm rule later displays Θ_s explicitly, because that rule uses the distinguished subterm U_s and target R_i. For a chosen profile class

$$\omega = (\vec{\alpha}, \vec{\beta}; \gamma) \in \Sigma_s,$$

where $\vec{\alpha}$ records the sorts of the index variables and $\vec{\beta}$ records the sorts of the rely variables, the generated modal type former is

$$\langle K_s \rangle^{\vec{x}::\vec{A}}_{\vec{y}::\vec{B}} C.$$

The notation for contextual modalities is analogous to the notation for dependent products. The generated dependent product constructor is a morphism

$$\Pi : \mathrm{Pr} \times \mathrm{Pr}^{\mathrm{Pr}} \longrightarrow \mathrm{Pr},$$

and the displayed type

$$\Pi_{y::B} C$$

is syntactic sugar for

$$\Pi(B, \lambda y.C).$$

Similarly, a modal site s, with context $K_s[-]$, generates an actual constructor in the operational theory; the displayed notation

$$\langle K_s \rangle^{\vec{x}::\vec{A}}_{\vec{y}::\vec{B}} C$$

is sugar for applying that constructor to the index types, the rely-type family, and the target-type family. The displayed notation names the typed interface. In formal rules the matching carrier-and-evidence telescope is written explicitly:

$$\Gamma, \vec{x} : \vec{X}_s, \vec{y} : \vec{Y}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A}, \vec{e}_y : \vec{y} :: \vec{B}.$$

Index variables are written in the superscript and come first, while rely variables are written in the subscript and are discharged by the modal type former. Since the rely types may depend on the index variables, the judgments for $\vec{B}$ are formed after the carrier declarations and evidence assumptions for $\vec{x}$, and the target type C is formed after both the index and rely declarations. If ambient variables are needed in a particular instance, they are included in the outer carrier context Γ, together with any ambient evidence assumptions in Δ.

Let $\vec{\aleph}$ be the product of the carriers of the index variables $\vec{x}$, and let $\vec{\beth}$ be the product of the carriers of the rely variables $\vec{y}$. The modal constructor associated to s has signature

$$\langle K_s \rangle : \vec{\aleph} \times \vec{\beth}^{\vec{\aleph}} \times \mathrm{Pr}^{\vec{\aleph} \times \vec{\beth}} \longrightarrow \mathrm{Pr}^{\vec{\aleph}}.$$

Given index types $\vec{A} : \vec{\aleph}$, rely types depending on the index variables,

$$\lambda\vec{x}.\vec{B} : \vec{\beth}^{\vec{\aleph}},$$

and a target type family

$$\lambda(\vec{x},\vec{y}).C : \mathrm{Pr}^{\vec{\aleph}\times\vec{\beth}},$$

the displayed modal type is the resulting Pr-term in the index context, over whatever outer context Γ is present:

$$\langle K_s\rangle^{\vec{x}::\vec{A}}_{\vec{y}::\vec{B}} C \;:=\; \langle K_s\rangle\big(\vec{A}, \lambda\vec{x}.\vec{B}, \lambda(\vec{x},\vec{y}).C\big)(\vec{x}).$$

When there are no index variables, $\vec{\aleph}$ is terminal and the codomain $\mathrm{Pr}^{\vec{\aleph}}$ is just Pr. In the lambda-calculus case, the application site has no index variables and one rely variable of carrier Pr, so this specializes to

$$\langle \mathsf{app}([-], y)\rangle : \mathrm{Pr} \times \mathrm{Pr}^{\mathrm{Pr}} \longrightarrow \mathrm{Pr},$$

which is exactly the dependent-product constructor.

The index variables $\vec{x}$ remain in the external context. The rely variables $\vec{y}$, and their evidence variables $\vec{e}_y$, are discharged by the modal type former:

$$\frac{\begin{array}{c}\Gamma \mid \Delta \vdash \vec{A} :: \vec{\alpha} \qquad \Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash \vec{B} :: \vec{\beta} \\ \Gamma, \vec{x} : \vec{X}_s, \vec{y} : \vec{Y}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A}, \vec{e}_y : \vec{y} :: \vec{B} \vdash C :: \gamma\end{array}}{\Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash \langle K_s\rangle^{\vec{x}::\vec{A}}_{\vec{y}::\vec{B}} C :: \gamma}\ \textsc{modal-form.}$$

Here $\Gamma \mid \Delta \vdash \vec{A} :: \vec{\alpha}$ abbreviates the telescope of judgments saying that each index type has the corresponding sort. The premise

$$\Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash \vec{B} :: \vec{\beta}$$

abbreviates the corresponding telescope for the rely types in the index carrier-and-evidence context; in particular, a rely type may mention variables already in Γ and the index carrier variables, but not the discharged rely variables themselves.

Modal introduction:

$$\frac{\begin{array}{c}\Gamma, \vec{x} : \vec{X}_s, \vec{y} : \vec{Y}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A}, \vec{e}_y : \vec{y} :: \vec{B} \vdash r : K_s[P] \rightsquigarrow Q \\ \Gamma, \vec{x} : \vec{X}_s, \vec{y} : \vec{Y}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A}, \vec{e}_y : \vec{y} :: \vec{B} \vdash Q :: C\end{array}}{\Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash P :: \langle K_s\rangle^{\vec{x}::\vec{A}}_{\vec{y}::\vec{B}} C}\ \textsc{modal-intro.}$$

In the premises, the outer context and index variables are already fixed and P is weakened along the discharged rely carrier variables and their explicit evidence assumptions. The conclusion then discharges $\vec{y} : \vec{Y}_s$ and $\vec{e}_y : \vec{y} :: \vec{B}$, leaving the external context

$$\Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A}.$$

Modal elimination:

$$\frac{\Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash P :: \langle K_s\rangle^{\vec{x}::\vec{A}}_{\vec{y}::\vec{B}} C \qquad \Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash \vec{b} :: \vec{B}}{\Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash K_s[P][\vec{b}/\vec{y}] :: \Diamond\big(C[\vec{b}/\vec{y}]\big)}\ \textsc{modal-elim.}$$

In the second premise, the notation $\vec{b} :: \vec{B}$ means that the rule is supplied with carrier arguments $\vec{b}$ together with derivations of the corresponding rely typings. These premise derivations are the evidence fed to the contextual-modality evidence.

The primitive rewrite constructor itself supplies a useful derived introduction rule. If $s = (i, o)$ and $L_i = K_s[U_s]$, then, in the ambient, index, and rely context, ρ_i gives

$$\Gamma, \Theta_s, \vec{x} : \vec{X}_s, \vec{y} : \vec{Y}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A}, \vec{e}_y : \vec{y} :: \vec{B} \vdash \rho_i : K_s[U_s] \rightsquigarrow R_i.$$

Therefore modal introduction yields

$$\frac{\Gamma, \Theta_s, \vec{x} : \vec{X}_s, \vec{y} : \vec{Y}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A}, \vec{e}_y : \vec{y} :: \vec{B} \vdash R_i :: C}{\Gamma, \Theta_s, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash U_s :: \langle K_s \rangle^{\vec{x} :: \vec{A}}_{\vec{y} :: \vec{B}} C} \text{ SOURCE-SUBTERM-INTRO.}$$

This is the rule that turns a declared source site of a rewrite into a contextual may-precondition type. The ambient telescope Θ_s is essential: it accounts for the free variables of U_s that are not supplied by $K_s[-]$, as well as any primitive-rule parameters needed to form the target. Its premise uses the same index-first order as modal introduction, with the ambient telescope placed before the index declarations, because the target type may depend on ambient variables, shared indices, and supplied rely variables. Ambient carrier declarations live in Θ_s; any ambient evidence assumptions live in Δ.

Computation for canonical source introductions. When rules and their evidence are retained proof-relevantly, the generated presentation includes the computation equation saying that source-subterm introduction followed by modal elimination is the one-step possibility proof obtained from the displayed primitive rewrite. Let d be a derivation of

$$\Gamma, \Theta_s, \vec{x} : \vec{X}_s, \vec{y} : \vec{Y}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A}, \vec{e}_y : \vec{y} :: \vec{B} \vdash R_i :: C,$$

and let $\mathrm{intro}^{\rho_i}_s(d)$ denote the corresponding source-subterm introduction derivation. If $\vec{b}$ is a tuple of rely arguments, supplied together with rely-typing evidence, then

$$\mathrm{elim}_s\big(\mathrm{intro}^{\rho_i}_s(d), \vec{b}\big) = \Diamond\text{-}\,\mathrm{intro}\big(\rho_i[\vec{b}/\vec{y}], d[\vec{b}/\vec{y}]\big).$$

The two sides are derivations of the same judgment

$$\Gamma, \Theta_s, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash K_s[U_s][\vec{b}/\vec{y}] :: \Diamond\big(C[\vec{b}/\vec{y}]\big).$$

Thus, for the canonical source subterm, the generated elimination rule computes to the original operational rewrite constructor. Variables or other terms of modal type may still be eliminated: their modal typing assumption is interpreted as abstract evidence of the same interface, rather than as a syntactic claim that the term literally has source shape U_s.

9 Interaction of $\Diamond$ with contextual modalities

The possibility operator is a one-step operational effect, not a type-theoretic counit. The completion therefore does not contain a rule

$$\frac{\Gamma \mid \Delta \vdash P :: \Diamond C}{\Gamma \mid \Delta \vdash P :: C} \ .$$

Such a rule would collapse may-reachability into actual membership and would amount to a form of subject expansion. The typed term calculus used in the lambda-cube comparison is instead the constructor fragment: typed terms are built from typed subterms by syntax-directed constructor rules. The full modal rules above remain available as a proof-theoretic layer for may-properties, but unrestricted rewrite evidence is not a term former of the typed programming calculus.

The interaction between $\Diamond$ and a contextual modality is *admissible delayed* elimination, and this admissible principle is available only when the relevant site context has explicit closure support. A judgment $P :: \Diamond C$ says that P itself has a one-step successor of type C. It does *not* say that $K[P]$ can step inside every surrounding context $K[-]$. To move that step through a particular site context one needs declared closure data $\kappa_s \Vdash K_s[-]$ in this sense.

Nested diamonds record the number of operational steps that have been postponed. Write

$$\Diamond^0 C = C, \qquad \Diamond^{n+1} C = \Diamond(\Diamond^n C).$$

For $n = 0$, modal elimination says that a term of contextual type can be placed in its operational context and the result has type $\Diamond C$. No contextual-closure rule is needed in this case, because the step is the site interaction itself. For $n > 0$, a derivation of $P :: \Diamond^n M$ contains delayed rewrite evidence before the site interaction can occur. Each of those preliminary steps must be transported through $K_s[-]$ by an explicit closure support.

Concretely, for $n > 0$, delayed elimination is not added to the generated presentation as a primitive rule. It is the following admissible principle, available only when $\kappa_s \Vdash K_s[-]$:

Proposition 9.1 (Admissible delayed modal elimination)**.** *For every $n > 0$, if the site s has closure support $\kappa_s \Vdash K_s[-]$, then the following rule is admissible:*

$$\frac{\kappa_s \Vdash K_s[-] \qquad \begin{array}{c}\Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash P :: \Diamond^n\big(\langle K_s \rangle^{\vec{x} :: \vec{A}}_{\vec{y} :: \vec{B}} C\big) \\ \Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash \vec{b} :: \vec{B}\end{array}}{\Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash K_s[P][\vec{b}/\vec{y}] :: \Diamond^{n+1}\big(C[\vec{b}/\vec{y}]\big)}\ \text{ADM. DELAYED-MODAL-ELIM.}\ .$$

Proof sketch. The proof is by induction on the displayed nested $\Diamond$-evidence in the premise. For $n = 1$, the premise gives a one-step witness from P to some P' with contextual type $\langle K_s \rangle^{\vec{x} :: \vec{A}}_{\vec{y} :: \vec{B}} C$. Closure support transports that witness to a step $K_s[P][\vec{b}/\vec{y}] \rightsquigarrow K_s[P'][\vec{b}/\vec{y}]$, and ordinary modal elimination applied to P' gives $K_s[P'][\vec{b}/\vec{y}] :: \Diamond(C[\vec{b}/\vec{y}])$. A single use of $\Diamond$-introduction therefore gives the required $\Diamond^2(C[\vec{b}/\vec{y}])$. The successor step repeats the same transport through $K_s[-]$ before applying the induction hypothesis. Thus the unbounded family above is a metatheorem about nested possibility evidence, not an infinite family of primitive rules in $\mathcal{H}(A)$. □

The premise $\kappa_s \Vdash K_s[-]$ is a side condition on the operational presentation, not a consequence of the typing premise. The successor case says: if P may take n steps to a process with contextual type, then plugging P into the site context may first propagate those delayed steps through $K_s[-]$, and then perform the site interaction itself. The result is not a term of type C; it is a term of type $\Diamond^{n+1} C$. Thus the admissible principle composes may-information without ever erasing it. In a proof-relevant presentation, a derivation of the premise carries the staged rewrite evidence being propagated. In a proof-irrelevant predicate interpretation, the same admissible principle uses the corresponding existential image and therefore belongs to the regular structure used to interpret may-reachability, not to a counit $\Diamond C \to C$.

For example, suppose the presentation contains left-parallel closure

$$r : P \rightsquigarrow P' \quad \longmapsto \quad \mathsf{par}_1(r) : P \mid q \rightsquigarrow P' \mid q$$

and the structural commutativity equation

$$P \mid q = q \mid P.$$

Then closure support for the right-hole context $q \mid [-]$ is obtained by congruence transport:

$$q \mid P = P \mid q \rightsquigarrow P' \mid q = q \mid P'.$$

Thus $q \mid [-]$ need not be declared as a separate primitive closure rule. What commutativity does not do is create a new displayed site address or a new profile quotient. It only transports rewrite evidence that is already available from the displayed closure rule.

This also explains how $\Diamond$ interacts with products in the lambda instance. For $n = 0$, ordinary modal elimination gives

$$f :: \Pi(A, \lambda x.B), \quad a :: A \quad \Longrightarrow \quad \mathsf{app}(f, a) :: \Diamond(B[a/x]).$$

For $n > 0$, because the application context is closure-supported by the usual head context rule, product elimination extends to

$$\frac{\mathsf{head} \Vdash \mathsf{app}([-], a) \qquad \Gamma \mid \Delta \vdash f :: \Diamond^n\big(\Pi(A, \lambda x.B)\big) \qquad \Gamma \mid \Delta \vdash a :: A}{\Gamma \mid \Delta \vdash \mathsf{app}(f, a) :: \Diamond^{n+1}\big(B[a/x]\big)} \text{ ADM. DELAYED-}\Pi\text{-ELIM.}.$$

In particular, from $f :: \Diamond(\Pi(A, \lambda x.B))$ and $a :: A$ one gets

$$\mathsf{app}(f, a) :: \Diamond^2\big(B[a/x]\big),$$

not $\mathsf{app}(f, a) :: B[a/x]$. This is the desired behavior: the typing judgment remembers that one step is needed to reach a function-like process and a second step is needed to perform the application. Without the explicit head closure support, the admissible delayed application principle is not available.

Proposition 9.2 (Admissible delayed contextual elimination)**.** *Assume $\kappa_s \Vdash K_s[-]$. For every $n > 0$, the following rule is admissible:*

$$\frac{\Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash P :: \Diamond^n\Big(\langle K_s \rangle_{\vec{y} :: \vec{B}}^{\vec{x} :: \vec{A}} C\Big) \qquad \Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash \vec{b} :: \vec{B}}{\Gamma, \vec{x} : \vec{X}_s \mid \Delta, \vec{e}_x : \vec{x} :: \vec{A} \vdash K_s[P][\vec{b}/\vec{y}] :: \Diamond^{n+1}\Big(C[\vec{b}/\vec{y}]\Big)}.$$

Proof sketch. For $n = 1$, eliminate the hypothesis

$$P :: \Diamond\left(\langle K_s \rangle_{\vec{y} :: \vec{B}}^{\vec{x} :: \vec{A}} C\right)$$

to obtain fresh data

$$Q : \mathrm{Pr}, \qquad r : P \rightsquigarrow Q, \qquad Q :: \langle K_s \rangle_{\vec{y} :: \vec{B}}^{\vec{x} :: \vec{A}} C.$$

Closure support transports the rewrite through the site context:

$$\kappa_s(r) : K_s[P] \rightsquigarrow K_s[Q].$$

Modal elimination for Q gives

$$K_s[Q][\vec{b}/\vec{y}] :: \Diamond\left(C[\vec{b}/\vec{y}]\right).$$

Now $\Diamond$-introduction along $\kappa_s(r)$ gives

$$K_s[P][\vec{b}/\vec{y}] :: \Diamond^2\left(C[\vec{b}/\vec{y}]\right).$$

The case $n+1$ is identical after eliminating the outer $\Diamond$ and applying the induction hypothesis to the opened reduct. □

10 Conversion

The completion does not identify types along operational rewrites. In particular, it does not include a rule of the form

$$\frac{\Gamma \mid \Delta \vdash P :: A \qquad \Gamma \mid \Delta \vdash r : A \rightsquigarrow B \qquad \Gamma \mid \Delta \vdash B :: u}{\Gamma \mid \Delta \vdash P :: B}$$

for arbitrary rewrites r. Such a rule is appropriate only when the chosen rewrite relation is intended to be definitional equality at the type level, as in some presentations of beta-conversion for dependent lambda calculi. It is not appropriate for a general operational theory, and especially not for concurrent calculi.

The only conversion principle present in the base construction is ordinary equational substitution. If the presentation proves an equation $A = B$, then typing may be transported across that equation:

$$\frac{\Gamma \mid \Delta \vdash P :: A \qquad \Gamma \mid \Delta \vdash A = B}{\Gamma \mid \Delta \vdash P :: B}$$

This is just the usual stability of judgments under the equations of the underlying presentation of the operational theory.

A particular instance may add a separate type-conversion relation $\equiv$, together with a rule

$$\frac{\Gamma \mid \Delta \vdash P :: A \qquad \Gamma \mid \Delta \vdash A \equiv B \qquad \Gamma \mid \Delta \vdash B :: u}{\Gamma \mid \Delta \vdash P :: B}$$

but this is extra structure.

Likewise, subject reduction is not assumed by the generic construction. A calculus may satisfy a theorem saying that $P :: A$ and $P \rightsquigarrow Q$ imply $Q :: A$, or a suitable evolution of A, but this is an additional property of a particular typed operational theory. The modal introduction rule instead records the target typing explicitly.

11 Church-style annotation of binding constructors

The completion follows Barendregt's Church-style convention: constructors that bind variables are refined so that the type of the bound variable is part of the raw term. The guiding principle is:

> If a constructor discharges a typed variable in its generated typing rule, then the constructor receives an explicit type annotation for that variable.

Suppose the input presentation has a binding constructor

$$c : Z \times \mathrm{Pr}^X \to \mathrm{Pr}.$$

The completion contains a Church-style refinement

$$c^\sharp : Z \times X \times \mathrm{Pr}^X \to \mathrm{Pr}.$$

The additional argument is the type annotation for the bound variable of carrier X. In this generic same-carrier formulation, an X-variable is typed by another element of X; proof-relevantly, the

generated typing display for that carrier lies over $X \times X$. Thus, in the π-calculus example, a name may also serve as a name-type or name-classifier. A more refined presentation could replace the second X by a separate type-object Ty_X, but the present construction keeps the annotation carrier identical to the variable carrier. More generally, a constructor binding variables of carriers $X_1, \ldots, X_m$ receives one type argument in each corresponding carrier.

There is an erasure map on raw syntax

$$\mathsf{erase}(c^\sharp(z, A, k)) = c(z, k).$$

The annotation is operationally irrelevant but syntactically relevant. Primitive rewrite constructors are lifted to their annotated versions by carrying the annotations through the source and target while erasing to the original rewrite.

For lambda abstraction, the raw constructor

$$\mathsf{lam} : \mathrm{Pr}^{\mathrm{Pr}} \to \mathrm{Pr}$$

is refined to

$$\mathsf{lam}^\sharp : \mathrm{Pr} \times \mathrm{Pr}^{\mathrm{Pr}} \to \mathrm{Pr}.$$

The beta rule is lifted to

$$A : \mathrm{Pr}, \quad t : \mathrm{Pr}^{\mathrm{Pr}}, \quad y : \mathrm{Pr} \vdash \beta_A : \mathsf{app}(\mathsf{lam}^\sharp(A, t), y) \rightsquigarrow t(y).$$

For asynchronous input, the raw constructor

$$\mathsf{in} : \mathrm{Nm} \times \mathrm{Pr}^{\mathrm{Nm}} \to \mathrm{Pr}$$

is refined to

$$\mathsf{in}^\sharp : \mathrm{Nm} \times \mathrm{Nm} \times \mathrm{Pr}^{\mathrm{Nm}} \to \mathrm{Pr},$$

written $n?_A.k$. The communication rule is lifted to

$$n, m, A : \mathrm{Nm}, \quad k : \mathrm{Nm} \to \mathrm{Pr} \vdash \mathsf{comm}_A : n!m \mid n?_A k \rightsquigarrow k(m).$$

Non-binding constructors do not require annotations in the minimal Church-style lift. Application and output consume terms but do not bind them.

12 Completing the untyped λ-calculus

Start with the finite untyped lambda presentation with constructors

$$\mathsf{app} : \mathrm{Pr} \times \mathrm{Pr} \to \mathrm{Pr}, \qquad \mathsf{lam} : \mathrm{Pr}^{\mathrm{Pr}} \to \mathrm{Pr},$$

primitive beta rewrite

$$t : \mathrm{Pr}^{\mathrm{Pr}}, \quad y : \mathrm{Pr} \vdash \beta : \mathsf{app}(\mathsf{lam}(t), y) \rightsquigarrow t(y),$$

and the head contextual closure constructor

$$q : \mathrm{Pr}, \quad r : \mathrm{R} \vdash \mathsf{head} : \mathsf{app}(s(r), q) \rightsquigarrow \mathsf{app}(t(r), q).$$

The head rule is not a product site; it is the explicit closure support for the application context. Use the product-site specification, which declares only the function-position occurrence

$$U = \mathsf{lam}(t)$$

in the beta source. The corresponding displayed context is

$$K[-] = \mathsf{app}([-], y).$$

The variable y in this context is a rely variable and there are no index variables. The abstraction body t is ambient: it occurs in the selected subterm $\mathsf{lam}(t)$, not in the surrounding application context. In the familiar lambda rule this ambient parameter is the schematic body of the abstraction, so it is displayed as the term typed in the premise rather than as a new modal axis. There is nothing to mod out by, so the profile space for this site is $\{*, \Box\}^2$. The argument occurrence and the whole beta source are not declared product sites, so this completion does not silently add argument-position or identity contextual modalities.

After Church refinement, the generated dependent product is notation for the corresponding contextual modality:

$$\Pi(A, \lambda y.B) := \langle \mathsf{app}([-], y) \rangle_{y::A} B.$$

For a chosen profile (u, v) in the selected subset Σ, the formation rule is

$$\frac{\Gamma \mid \Delta \vdash A :: u \qquad \Gamma, y : \mathrm{Pr} \mid \Delta, e_y : y :: A \vdash B :: v}{\Gamma \mid \Delta \vdash \Pi(A, \lambda y.B) :: v}\ \Pi\text{-FORM.}$$

The abstraction rule is generated by the lifted beta rewrite and modal introduction:

$$\frac{\Gamma \mid \Delta \vdash A :: u \qquad \Gamma, y : \mathrm{Pr} \mid \Delta, e_y : y :: A \vdash B :: v \qquad \Gamma, y : \mathrm{Pr} \mid \Delta, e_y : y :: A \vdash t :: B}{\Gamma \mid \Delta \vdash \mathsf{lam}^\sharp(A, \lambda y.t) :: \Pi(A, \lambda y.B)}\ \Pi\text{-INTRO.}$$

Indeed, under $y : \mathrm{Pr} \mid e_y : y :: A$, beta gives

$$\mathsf{app}(\mathsf{lam}^\sharp(A, \lambda y.t), y) \rightsquigarrow t,$$

and the third premise gives $t :: B$. Modal introduction therefore gives

$$\Gamma \mid \Delta \vdash \mathsf{lam}^\sharp(A, \lambda y.t) :: \langle \mathsf{app}([-], y) \rangle_{y::A} B.$$

Application is modal elimination:

$$\frac{\Gamma \mid \Delta \vdash f :: \Pi(A, \lambda x.B) \qquad \Gamma \mid \Delta \vdash a :: A}{\Gamma \mid \Delta \vdash \mathsf{app}(f, a) :: \Diamond\big(B[a/x]\big)}\ \Pi\text{-ELIM.}$$

or, using the built-in lambda syntax,

$$\frac{\Gamma \mid \Delta \vdash f :: \Pi(A, k) \qquad \Gamma \mid \Delta \vdash a :: A}{\Gamma \mid \Delta \vdash \mathsf{app}(f, a) :: \Diamond\big(k(a)\big)}\ \Pi\text{-ELIM.}$$

So a function is a term that, when placed in the application context with an argument of type A, can take one rewrite step to a result of type B. Because the lambda presentation includes the head contextual closure constructor, Proposition 9.1 also gives, for $n > 0$, the admissible delayed form

$$\frac{\mathsf{head} \Vdash \mathsf{app}([-], a) \qquad \Gamma \mid \Delta \vdash f :: \Diamond^n\big(\Pi(A, \lambda x.B)\big) \qquad \Gamma \mid \Delta \vdash a :: A}{\Gamma \mid \Delta \vdash \mathsf{app}(f, a) :: \Diamond^{n+1}\big(B[a/x]\big)}\ \text{ADM. DELAYED-}\Pi\text{-ELIM.}$$

Thus a delayed function may still be applied by an admissible metarule, but each delayed operational step remains visible as an additional $\Diamond$.

If the operational presentation is extended with an argument-position closure rule

$$q : \mathrm{Pr},\ r : \mathrm{R} \vdash \mathsf{arg}(q, r) : \mathsf{app}(q, s(r)) \rightsquigarrow \mathsf{app}(q, t(r)),$$

then the argument-hole context $\mathsf{app}(q, [-])$ has closure support. Hence, in the nondependent case, there is an admissible strict delayed-argument principle

$$\frac{f :: A \to B \qquad g :: \Diamond^n A}{\mathsf{app}(f, g) :: \Diamond^{n+1} B}.$$

This principle describes one possible reduction path: first reduce the argument until it has type A, then apply f. It is not a precise normal-order cost rule. Adding arg enlarges the reduction relation beyond pure normal-order reduction, and since $\Diamond$ is a may-modality, the result records existence of such a path rather than the least number of normal-order steps.

Moreover, no rule depending only on

$$f :: A \to B \qquad g :: \Diamond A$$

can determine the precise normal-order delay. Functions with the same type may discard, use, or duplicate their argument. For example, if $a :: A$, then

$$\mathsf{app}(\lambda x.a, g) \rightsquigarrow a$$

gives a one-step possibility, while

$$\mathsf{app}(\lambda x.x, g) \rightsquigarrow g$$

and $g :: \Diamond A$ give a two-step possibility. Thus precise delay accounting requires additional usage or cost structure; it is not determined by the ordinary function type alone.

12.1 The lambda-cube geometry

Since we require $(*, *)$ as a base profile, the remaining three optional profiles form the usual three-dimensional Lambda cube.

This is a recovery of the cube's sort geometry and constructor fragment, not an assertion that the completed modal calculus has exactly the same judgments as a pure type system. The application rule is effectful: its result is $\Diamond(B[a/x])$, and delayed applications produce further nested diamonds. For comparison with Barendregt's systems we use the constructor-fragment translation of Section 16, which erases $\Diamond$. We deliberately do not add a rule $\Diamond A \Rightarrow A$; that rule would type terms by subject expansion and would let arbitrary may-reachability masquerade as ordinary membership.

13 Completing the asynchronous π-calculus

For the asynchronous communication rule

$$n, m : \mathrm{Nm}, \quad k : \mathrm{Nm} \to \mathrm{Pr} \vdash \mathsf{comm} : n!m \mid n?k \rightsquigarrow k(m),$$

the displayed source is ordered with the output on the left and the input on the right. This order is display data, even though the structural equations later identify it with the reversed parallel composite. Use the receive-site specification, which declares the input occurrence

$$U = n?k.$$

The corresponding context is

$$K[-] = n!m \mid [-].$$

This context is extracted from the displayed source tree. For any plugged process P, commutativity gives $n!m \mid P = P \mid n!m$, but that equation does not create a second generated modality. The variable n appears both in U and in K, so it is an index variable. The variable m appears in K but not in U, so it is a rely variable. The continuation k appears in the subterm and in the target $k(m)$, but not in the surrounding context, so it is an ambient variable:

$$\vec{x} = (n), \qquad \vec{y} = (m), \qquad \vec{z} = (k), \qquad \Theta_{\mathsf{recv}} = (k : \mathrm{Nm} \to \mathrm{Pr}).$$

The declaration for k is displayed as a structural higher-order declaration; if the continuation variables themselves carry a typed discipline, that additional information belongs in the same ambient telescope.

After Church refinement, input is written

$$\mathsf{in}^{\sharp}(n, B, k),$$

or with syntactic sugar as

$$n?_B k,$$

where B classifies the received message. For a chosen profile assigning a sort to the index type A of the channel n, the rely type B of the message m, and the target type C, the generated input rule uses an ambient and index variable context

$$\Gamma, \Theta_{\mathsf{recv}}, n : \mathrm{Nm}, m : \mathrm{Nm}$$

together with explicit relation-context evidence assumptions

$$\Delta, e_n : n :: A, e_m : m :: B.$$

The rely variable m and its evidence assumption $e_m : m :: B$ are discharged by the modal type former, while the index variable n and its evidence assumption $e_n : n :: A$ remain in the external context.

$$\frac{\Gamma, \Theta_{\mathsf{recv}}, n : \mathrm{Nm}, m : \mathrm{Nm} \mid \Delta, e_n : n :: A, e_m : m :: B \vdash k(m) :: C}{\Gamma, \Theta_{\mathsf{recv}}, n : \mathrm{Nm} \mid \Delta, e_n : n :: A \vdash n?_B k :: \langle n!m \mid [-] \rangle^{n::A}_{m::B} C} \text{ INPUT.}$$

The elimination rule keeps the same ambient continuation parameter:

$$\frac{\begin{array}{c}\Gamma, \Theta_{\mathsf{recv}}, n : \mathrm{Nm} \mid \Delta, e_n : n :: A \vdash P :: \langle n!m \mid [-] \rangle^{n::A}_{m::B} C \\ \Gamma, \Theta_{\mathsf{recv}}, n : \mathrm{Nm} \mid \Delta, e_n : n :: A \vdash a :: B\end{array}}{\Gamma, \Theta_{\mathsf{recv}}, n : \mathrm{Nm} \mid \Delta, e_n : n :: A \vdash n!a \mid P :: \Diamond\big(C[a/m]\big)} \text{ INPUT-ELIM.}$$

Thus an input process has a type describing the operational guarantee that, when paired with a suitable output, it has a one-step possibility of reaching a continuation of type C.

The output occurrence $n!m$ may be declared as a separate site to generate the dual output-context modality, but it is not part of the receive-site hypercube below; it would produce its own hypercube.

13.1 The receive hypercube.

For the receive site s_{recv}, no consistency relation identifies or rules out any profiles, so the carrier-sensitive profile set is exactly

$$\{*^{\mathrm{Nm}}, \square^{\mathrm{Nm}}\}_n \times \{*^{\mathrm{Nm}}, \square^{\mathrm{Nm}}\}_m \times \{*^{\mathrm{Pr}}, \square^{\mathrm{Pr}}\}_C.$$

When the carriers are clear, we abbreviate this as $\{*, \square\}^3$. Since we require the base profile $(*^{\mathrm{Nm}}, *^{\mathrm{Nm}}; *^{\mathrm{Pr}})$, then the remaining seven profiles are optional and there are $2^7 = 128$ signature choices. A profile $(\alpha, \beta; \gamma) \in \Sigma \subseteq \{*, \square\}^3$, with carrier superscripts suppressed when unambiguous, should be read as follows: α is the sort of the channel type A for the indexed channel n, β is the sort of the message type B for the received name m, and γ is the sort of the continuation target type C. The base profile $(*^{\mathrm{Nm}}, *^{\mathrm{Nm}}; *^{\mathrm{Pr}})$ gives first-order receive: ordinary channels receive ordinary messages and continue at an ordinary process type. The profile $(*^{\mathrm{Nm}}, *^{\mathrm{Nm}}; \square^{\mathrm{Pr}})$ is the receive analogue of dependent types: a type-level protocol family may depend on ordinary channel and message values. Profiles with $\gamma = *^{\mathrm{Pr}}$ and at least one of α, β equal to $\square^{\mathrm{Nm}}$ are polymorphic receive principles: ordinary continuations may be indexed by type-level channels, by type-level payloads, or by both. Profiles with $\gamma = \square^{\mathrm{Pr}}$ and at least one of α, β equal to $\square^{\mathrm{Nm}}$ are receive type-operator principles: protocol families may themselves be parameterized by type-level channels, type-level payloads, or both. Finally, because the message type B is formed in the context $n :: A$, the pair (α, β) also controls the dependency of the message classifier on the channel classifier: $(*^{\mathrm{Nm}}, *^{\mathrm{Nm}})$ is the ordinary first-order case, $(*^{\mathrm{Nm}}, \square^{\mathrm{Nm}})$ allows channel-dependent message classifiers, and profiles with $\alpha = \square^{\mathrm{Nm}}$ allow the channel itself to be indexed by type-level name data.

14 Freeness and functoriality

The construction is not merely a recipe. It is free: once the displayed sites, profile signatures, and closure-support annotations have been chosen, no hidden typing structure is imposed. To interpret the completed system in another typed operational theory is exactly to interpret the underlying site-and-signature data, together with the chosen generated structure.

The precise categories are defined in Appendix C. Here **PSig** is the category of finite site-and-signature presentations and strict morphisms preserving displayed source addresses, matching data, signatures, and closure support. The target category **PDTOT** consists of typed operational presentations equipped with a specified interpretation of the generated hypercube structure. The forgetful functor

$$U : \mathbf{PDTOT} \to \mathbf{PSig}$$

forgets that interpretation.

Theorem 14.1 (Free hypercube completion)**.** *The structured completion functor*

$$\mathsf{Hyp} : \mathbf{PSig} \to \mathbf{PDTOT}$$

is left adjoint to U. Equivalently, for every $A \in \mathbf{PSig}$ and every structured typed operational theory $X \in \mathbf{PDTOT}$, there is a natural bijection

$$\mathrm{Hom}_{\mathbf{PDTOT}}(\mathsf{Hyp}(A), X) \cong \mathrm{Hom}_{\mathbf{PSig}}(A, U(X)).$$

Proof sketch. A morphism out of $\mathsf{Hyp}(A)$ is forced by its action on the underlying site-and-signature presentation A. The sorts, typing evidence displays, modal constructors, source-subterm

rules, and computation equations of $\mathsf{Hyp}(A)$ are freely generated from that data. Closure-support annotations are part of the underlying site data; they justify admissible delayed elimination but do not add primitive delayed rules. Conversely, any strict morphism $A \to U(X)$ extends uniquely by applying the structure map of X to each generated symbol and rule. Thus the displayed hom-set map sends a structured morphism to its underlying **PSig**-morphism, and the inverse extends a **PSig**-morphism by freeness. The unit is the identity on A, and the counit is the specified structure map of X. The triangle identities reduce to identity laws. Appendix C contains the formal category definitions, functoriality proof, unit, counit, and triangle verification. □

15 Proof-relevant evidence semantics and soundness

The generated rules have a proof-relevant semantic reading. A typing judgment is not interpreted first as a truth-valued predicate; it is interpreted as an object of evidence. For each relevant carrier X, a structured model carries a display

$$p_X : \mathsf{Ev}_X \longrightarrow X \times X, \qquad p_X = \langle \mathsf{tm}_X, \mathsf{tp}_X \rangle,$$

not assumed to be monic. An element $e \in \mathsf{Ev}_X$ is evidence that $\mathsf{tm}_X(e)$ has classifier $\mathsf{tp}_X(e)$. Thus a derivation of $\Gamma \mid \Delta \vdash P :: A$ is interpreted by a morphism

$$e_P : [\![\Gamma \mid \Delta]\!] \longrightarrow \mathsf{Ev}_X$$

whose projection to $X \times X$ is $\langle [\![P]\!]_{\Gamma|\Delta}, [\![A]\!]_{\Gamma|\Delta} \rangle$.

Typed context extension is still interpreted by pullback. If $A : X$ is a type code in context $\Gamma \mid \Delta$, then

$$[\![\Gamma, x : X \mid \Delta, e_x : x ::^X A]\!] = ([\![\Gamma \mid \Delta]\!] \times X) \times_{X \times X} \mathsf{Ev}_X,$$

where the map $[\![\Gamma \mid \Delta]\!] \times X \to X \times X$ is $\langle \pi_2, [\![A]\!]\pi_1 \rangle$. A point of this pullback is a carrier element together with named evidence witnessing that it has the displayed type. Weakening and substitution are therefore ordinary pullback operations on these evidence displays.

With this convention, the modal rules retain their operational witnesses. Evidence for $P :: \Diamond C$ is one-step may-evidence: it supplies a reduct Q, a rewrite witness $r : P \rightsquigarrow Q$, and evidence that $Q :: C$. The $\Diamond$-introduction rule constructs this tuple, and the $\Diamond$-eliminator is its dependent eliminator. Similarly, evidence for a contextual modality is an interface for the declared site context: when supplied with rely arguments and their evidence, modal elimination returns $\Diamond$-evidence for the plugged process. Source-subterm introduction is sound because the primitive rewrite constructor itself supplies the required one-step witness.

Theorem 15.1 (Soundness)**.** *In every structured proof-relevant model of the generated typed operational presentation, every derivable typing judgment has semantic evidence.*

Proof sketch. The proof is by induction on derivations. Structural rules are interpreted by finite-limit operations on evidence contexts. Rules that discharge typed variables, such as modal introduction and Church-style abstraction, use the specified binding/exponential structure of the operational presentation, or equivalently the named rule morphisms supplied by a structured model. The $\Diamond$-introduction rule pairs a rewrite witness with evidence for the typed target. The $\Diamond$-eliminator destructs this proof-relevant evidence. Modal elimination applies the evidence interface carried by a contextual modality to supplied rely evidence. The admissible delayed-elimination principle is sound only for sites with explicit closure support; the closure constructor transports the staged rewrite evidence through the displayed context before the site interaction occurs. Appendix D gives the rule-by-rule verification. □

Proposition 15.2 (Proof-irrelevant may semantics)**.** *The proof-relevant soundness theorem does not require typing displays to be subobjects. If one forgets evidence and wants to interpret $\Diamond C$ and contextual modalities as proof-irrelevant may-predicates, then extra structure is required: regular images for one-step possibility, and dependent universal operations for contextual modalities that discharge rely variables.*

Proof sketch. Proof-relevant judgments retain rewrite and typing evidence. Proof-irrelevant may semantics forgets that evidence and asks only whether some evidence exists; this existential collapse is an image operation. For contextual modalities, the rely variables are abstracted out of the interface, so the proof-irrelevant reading also needs the corresponding dependent operation over those variables. Appendix D spells out this optional evidence-forgetting semantics. □

16 Constructor fragments and operational properties

The completion generates modal typing rules from operational sites. These rules have a direct may-reading, but they should not all be confused with syntax-directed term formation rules. In the typed programming language, the term calculus we use is the *constructor fragment*; the full completed system is a specification layer for may-properties of operational syntax.

By the constructor fragment we mean the subsystem in which typed terms are built only by the generated typed constructors and their syntax-directed rules. For the lambda-calculus completion, this includes the sort and structural rules, product formation, the $\mathsf{lam}^\sharp$-introduction rule, the app-elimination rule, and the admissible delayed application principles justified by the explicitly supplied head closure support. These admissible principles add no primitive typing rules to the generated presentation. The fragment does not include unrestricted $\Diamond$-introduction or modal-introduction applied to arbitrary raw terms. Thus every immediate object-language subterm of a typed process is itself typed by a premise of the rule that constructs it, even when the resulting type contains one or more $\Diamond$'s.

16.1 Termination in the λ-calculus

If a term is well-typed in one of the calculi of the lambda cube, it necessarily terminates. Consider the completion of the untyped lambda calculus in which we keep only the dependent-product-like site

$$K[-] = \mathsf{app}([-], x).$$

For a selected set of product profiles $\Sigma \subseteq \{*, \Box\}^2$ and for each $(u, v) \in \Sigma$, the constructor fragment has the following primitive rules, together with the displayed admissible delayed principle:

$$\frac{\Gamma \mid \Delta \vdash A :: u \qquad \Gamma, x : \mathrm{Pr} \mid \Delta, e_x : x :: A \vdash B :: v}{\Gamma \mid \Delta \vdash \Pi(A, \lambda x.B) :: v}\ \Pi\text{-FORM}$$

$$\frac{\Gamma \mid \Delta \vdash A :: u \qquad \Gamma, x : \mathrm{Pr} \mid \Delta, e_x : x :: A \vdash B :: v \qquad \Gamma, x : \mathrm{Pr} \mid \Delta, e_x : x :: A \vdash t :: B}{\Gamma \mid \Delta \vdash \mathsf{lam}^\sharp(A, \lambda x.t) :: \Pi(A, \lambda x.B)}\ \Pi\text{-INTRO}$$

$$\frac{\Gamma \mid \Delta \vdash f :: \Pi(A, \lambda x.B) \qquad \Gamma \mid \Delta \vdash a :: A}{\Gamma \mid \Delta \vdash \mathsf{app}(f, a) :: \Diamond\big(B[a/x]\big)}\ \Pi\text{-ELIM}$$

$$\frac{\mathsf{head} \Vdash \mathsf{app}([-], a) \qquad \Gamma \mid \Delta \vdash f :: \Diamond^n\big(\Pi(A, \lambda x.B)\big) \qquad \Gamma \mid \Delta \vdash a :: A}{\Gamma \mid \Delta \vdash \mathsf{app}(f, a) :: \Diamond^{n+1}\big(B[a/x]\big)}\ \text{ADM. DELAYED-}\Pi\text{-ELIM},\ n > 0$$

The Π-elimination rule is the generated modal elimination rule. The cases $n > 0$ are admissible delayed eliminations justified by the explicitly supplied head closure support, not primitive members of the generated presentation. The result type is therefore $\Diamond^{n+1}(B[a/x])$, not $B[a/x]$. Nevertheless, in the constructor fragment this difference does not introduce nonterminating untyped subterms.

Define a translation $(-)^\circ$ from types and terms in this fragment to the corresponding calculus in the lambda cube by erasing every possibility modality and by sending the generated product, application, and abstraction constructors to their counterparts:

$$(\Diamond^n C)^\circ = C^\circ,$$

$$\big(\Pi(A, \lambda x.B)\big)^\circ = \Pi_{x:A^\circ} B^\circ,$$

$$\big(\mathsf{app}(M, N)\big)^\circ = (M^\circ\ N^\circ),$$

$$\big(\mathsf{lam}^\sharp(A, \lambda x.t)\big)^\circ = \lambda x : A^\circ.\, t^\circ.$$

Proposition 16.1 (Termination in the lambda constructor fragment)**.** *Let P be a process term typable in the constructor fragment of the product-site completion of the untyped lambda calculus. Then P is strongly normalizing for beta-reduction.*

Proof sketch. The proof is by translation to the corresponding Barendregt system. By induction on the typing derivation, if

$$\Gamma \mid \Delta \vdash P :: A$$

is derivable in the constructor fragment, then

$$\Gamma^\circ \vdash P^\circ : A^\circ$$

is derivable in the corresponding pure type system. The only nontrivial case is the admissible delayed application principle. That principle concludes

$$\Gamma \mid \Delta \vdash \mathsf{app}(f, a) :: \Diamond^{n+1}\big(B[a/x]\big)$$

from premises $f :: \Diamond^n(\Pi(A, \lambda x.B))$ and $a :: A$. After erasing all diamonds, this becomes exactly the usual Barendregt application conclusion

$$\Gamma^\circ \vdash (f^\circ\ a^\circ) : B^\circ[a^\circ/x].$$

By the standard strong-normalization theorem for the lambda cube, in particular for the calculus of constructions [2, 3], every constructor-fragment subsystem obtained by choosing fewer product profiles is strongly normalizing. The translation preserves the computational beta-redexes, so termination of P° implies termination of P. If the operational presentation uses a sub-strategy such as normal-order beta reduction, this follows a fortiori from strong normalization for full beta reduction. □

The restriction to the constructor fragment is essential. The full modal completion also has rules that type a process from the existence of a rewrite witness. Those rules express may-behavior, but they are not syntax-directed term formation rules and can type processes whose subterms were not built from typed components. The termination statement above is therefore a statement about the lambda-cube-like fragment, not about the entire completed modal theory.

16.2 The analogous reading for the π-calculus

For a concurrent calculus, the analogous theorem is not necessarily a termination theorem. A process may be intended to run forever, wait for messages, participate in a race, or interact with an environment. The generated modal types instead describe one-step reachability properties.

Suppose $N : \mathrm{Pr}$ is a type intended to classify the nil process, and suppose the model or typing presentation contains

$$\frac{}{\Gamma \mid \Delta \vdash \mathrm{nil} :: N}\ \textsc{nil}.$$

If N is interpreted as the property of being structurally congruent to nil, then $P :: \Diamond N$ says that P may take one step to the nil process. More generally, the rule

$$\frac{\Gamma \mid \Delta \vdash r : P \rightsquigarrow Q \qquad \Gamma \mid \Delta \vdash Q :: N}{\Gamma \mid \Delta \vdash P :: \Diamond N}\ \Diamond\textsc{-intro}.$$

says that P has an explicit one-step witness to some N-typed target.

The receive site gives the contextual version of the same idea. From the communication rule

$$n!m \mid n?k \rightsquigarrow k(m),$$

the generated receive typing rule includes the instance, written with the ambient continuation telescope $\Theta_{\mathsf{recv}} = (k : \mathrm{Nm} \to \mathrm{Pr})$:

$$\frac{\Gamma, \Theta_{\mathsf{recv}}, n : \mathrm{Nm}, m : \mathrm{Nm} \mid \Delta, e_n : n :: A, e_m : m :: B \vdash k(m) :: N}{\Gamma, \Theta_{\mathsf{recv}}, n : \mathrm{Nm} \mid \Delta, e_n : n :: A \vdash n?_B k :: \langle n!m \mid [-] \rangle^{n::A}_{m::B} N}\ \textsc{recv-nil}.$$

Thus a process of type

$$\langle n!m \mid [-] \rangle^{n::A}_{m::B} N$$

is one which, when placed in parallel with an output $n!m$ supplying a message m with evidence $e_m : m :: B$ along an indexed channel n with evidence $e_n : n :: A$, may take one communication

step to a process of type N. If N is the nil property, this says that the process may communicate once and reach nil—but it may do something else!

The elimination rule makes this operational reading explicit:

$$\frac{\Gamma, \Theta_{\mathsf{recv}}, n : \mathrm{Nm} \mid \Delta, e_n : n :: A \vdash P :: \langle n!m \mid [-] \rangle^{n::A}_{m::B} N \qquad \Gamma, \Theta_{\mathsf{recv}}, n : \mathrm{Nm} \mid \Delta, e_n : n :: A \vdash a :: B}{\Gamma, \Theta_{\mathsf{recv}}, n : \mathrm{Nm} \mid \Delta, e_n : n :: A \vdash n!a \mid P :: \Diamond\big(N[a/m]\big)} \ \textsc{recv-elim}$$

When N does not depend on m, the conclusion is simply

$$n!a \mid P :: \Diamond N.$$

Nested modalities express staged may-reachability. For example, a type of the form

$$\langle K_2 \rangle^{x_2::A_2}_{y_2::B_2} \langle K_1 \rangle^{x_1::A_1}_{y_1::B_1} N$$

says that, after the outer index data are fixed and the outer context K_2 and its rely data are supplied, the process may step to a new process satisfying the inner modal type; after the inner index data are fixed and the inner context K_1 and its rely data are supplied, that continuation may step to a process of type N. If N denotes nil, this is a two-stage may-reach-nil property.

16.3 Must properties are model-theoretic

The modal types generated by the completion have an existential, proof-relevant shape. A derivation of

$$P :: \Diamond C$$

contains, or is generated from, a particular rewrite witness

$$r : P \rightsquigarrow Q$$

and a typing derivation

$$Q :: C.$$

Similarly, a derivation of

$$P :: \langle K \rangle^{\vec{x}::\vec{A}}_{\vec{y}::\vec{B}} C$$

records that, after the index variables are fixed and the rely variables are supplied, the plugged process $K[P]$ has a displayed one-step path to a C-typed target. Any ambient variables of the selected source subterm remain visible in the surrounding context throughout this derivation; they are not hidden in the modal interface. These are may-properties.

A must-property has a different quantifier shape. One-step must preservation would say something like

$$\forall Q.\ K[P] \rightsquigarrow Q \Longrightarrow Q \in [\![C]\!].$$

Such statements quantify over all rewrites in a model $\mathcal{M}$. They are therefore not determined by the syntactic modal typing rules alone.

This distinction matters especially for concurrent calculi. An arbitrary model of a presented operational theory may contain additional elements, additional rewrite witnesses, additional nondeterminism, or additional identifications beyond the freely generated syntax. A may-typing derivation is stable under such enlargement: the displayed witness still exists. A must-claim is

not stable in the same way, because adding a new possible rewrite can invalidate a universal statement about all rewrites.

The free model is the special case in which the only processes, equations, and rewrite witnesses are those generated by the presentation, modulo the specified equations. In the free model it can make sense to prove must-style metatheorems by induction on generated syntax or by operational analysis. Strong normalization for the lambda-cube constructor fragment is one such theorem. For a π-calculus fragment, analogous free-model theorems might say that every generated run respecting a closed protocol eventually reaches a process of type N, or that every generated communication step preserves some protocol invariant. But these are theorems about a chosen model or class of models, not inference rules supplied by the hypercube completion itself.

17 Relation to other presentations

Displayed evidence and subobject fibrations. The main semantics uses displayed evidence objects rather than mere truth-valued subobjects: a typing judgment is interpreted by a section of a display over the corresponding term-and-classifier pair. Finite limits provide the pullbacks needed for typed context extension, weakening, substitution, and the premise objects of generated rules. If one forgets evidence, the resulting proof-irrelevant predicates live in a subobject fibration [8, 9]; existential images, and for contextual modalities with discharged rely variables the corresponding dependent universal operations, are then needed for the optional proof-irrelevant interpretation of may-reachability.

Generalized algebraic theories. The generated rules are algebraic in the contextual sense: typed contexts are finite-limit objects, eliminators are morphisms out of those objects, and computation rules are equations. This is closely aligned with generalized algebraic theories and contextual categories [5].

Binding. The use of specified exponentials for continuations follows the abstract-syntax tradition in which binding structure is supplied as presentation data [6]. Because the exponential $\mathrm{Pr}^{\mathrm{Nm}}$ is specified in the asynchronous π-calculus presentation, its models are required to preserve the corresponding evaluation and substitution structure.

Reactive systems. Representing rule instances and contexts explicitly is compatible with the general lesson of reactive systems: operational behavior is organized by structured contexts and evidence, rather than by an unstructured binary relation alone [11].

18 Conclusion

The guiding observation of this paper is that dependent products are not isolated artifacts of lambda syntax. They are generated by a particular operational site: the function position of the beta redex. Once this is made explicit, the lambda cube becomes one instance of a more general construction. A displayed rewrite source supplies possible sites; a site supplies a one-hole context; and a choice of quotient sort profiles determines which modal dependent type formers are present.

The construction is deliberately presentation-sensitive. Displayed syntax names the sites, structural equations govern equality in the operational theory, and profile-matching data computes the quotient sort geometry. These layers are kept separate so that, for example, commutativity

of parallel composition does not silently move holes or merge input and output sites. Likewise, the possibility modality records one-step may-evidence; it does not include a counit $\Diamond A \Rightarrow A$, and delayed use of a modal type requires explicit contextual-closure support.

For the lambda-calculus, the application site recovers Barendregt's product geometry in the constructor fragment. For the asynchronous π-calculus, the communication rule gives different local cubes, with name and process carriers contributing their own sort levels. Thus the same recipe generates a family of typed operational systems from the redex structure of the language, rather than from lambda application alone.

The formal results support this recipe. The generated-presentation assignment is functorial, the structured completion is free in the sense of the adjunction proved in Appendix C, and the generated typing rules are sound in the proof-relevant evidence semantics proved in Appendix D. The main text has kept the examples and intuitions in front; the appendices contain the finite profile algorithm, the generated rule specification, and the full categorical and semantic verifications.

A Formal site-marked presentations and profile quotients

This appendix gives the formal version of the site and profile construction used in Section 4.3. As noted above, a site is a selected occurrence in the displayed source of a rewrite rule with carrier Pr that gets replaced with a hole to form a term context. When the selected subterm is placed into the context, a reduction may occur. The selection is deliberate: not every displayed Pr-subterm of a rewrite source should automatically become a modal type former. Sites are named before quotienting by structural equations; the completion is therefore presentation-sensitive in the same way that an operational semantics written with evaluation contexts is presentation-sensitive.

A.1 The fixed sorts

For each relevant carrier object X, the generated presentation adjoins constants

$$*^X : X, \qquad \Box^X : X,$$

for types and kinds, respectively, and a proof-relevant typing-evidence display

$$p_X : \mathsf{Ev}_X \longrightarrow X \times X, \qquad p_X = \langle \mathsf{tm}_X, \mathsf{tp}_X \rangle.$$

A point of Ev_X is evidence that the process or type code $\mathsf{tm}_X(e)$ has classifier $\mathsf{tp}_X(e)$. In contexts we write this evidence with a name, as $e : a ::^X b$. When the evidence name is immaterial in the conclusion of a rule, we may still write $a ::^X b$. The sort axiom is parametric in X:

$$\frac{}{\Gamma \mid \Delta \vdash *^X ::^X \Box^X} \text{ SORT.}$$

When the superscript is clear from the context, it is usually suppressed.

A.2 Displayed syntax, structural equations, and profile constraints

The construction is deliberately presentation-sensitive. It distinguishes the raw displayed syntax tree of a primitive rewrite source from the structural congruence generated by equations. A primitive rewrite constructor

$$\Gamma_i, \Delta_i \vdash \rho_i : L_i \rightsquigarrow R_i$$

therefore supplies a named displayed redex representative $L_i \rightsquigarrow R_i$. Sites are occurrence addresses in the raw tree L_i, before quotienting by any equations of the presentation.

Displayed equations have two different possible roles, and these roles are kept separate. First, they are ordinary equations of the operational theory: they generate structural congruence and support equational substitution in judgments. Second, an equation schema may carry explicit type-occurrence matching data $\mathsf{Match}_s(E)$, used only by the profile-consistency algorithm. The first role does not imply the second. In particular, an equation such as commutativity or associativity of a constructor does not automatically identify sites, replace a context by all congruent contexts, or identify the sort slots attached to moved occurrences.

Thus the following data are independent inputs to the completion:

(a) the displayed source representative L_i of each primitive rewrite constructor;

(b) the finite site specification $\mathfrak{S}_i$, whose elements are occurrence addresses in that displayed source;

(c) the displayed equations of the presentation, used as ordinary equations;

(d) the finite matching relations $\mathsf{Match}_s(E)$, used to generate profile-slot identifications.

Changing the displayed representative of a redex, adding an equivalent representative as a separate primitive rewrite, or adding explicit equation matchings can change the generated modal operators. Merely adding a structural equation changes the equational theory but not the set of sites extracted from a fixed source display.

A.3 Process sites

Let

$$\Gamma_i, \Delta_i \vdash \rho_i : L_i \rightsquigarrow R_i$$

be a primitive rewrite constructor in a finite presentation $\mathcal{P}$. Since L_i is a finite raw syntax tree, it has finitely many displayed subterm occurrences, even if L_i is structurally congruent to many other terms. Write ϵ_i for the root occurrence of L_i, and write

$$\mathsf{Occ}_{\mathrm{Pr}}(L_i)$$

for the finite set of occurrences whose displayed subterm has carrier Pr. Let

$$\mathsf{Occ}^{\mathrm{ch}}_{\mathrm{Pr}}(L_i) \subseteq \mathsf{Occ}_{\mathrm{Pr}}(L_i)$$

be the subset of proper occurrences $o \neq \epsilon_i$ whose displayed subterm is headed by an object-language constructor of $\mathcal{P}$. Variable occurrences and occurrences headed by the structural source and target maps $s, t : \mathrm{R} \to \mathrm{Pr}$ are not in $\mathsf{Occ}^{\mathrm{ch}}_{\mathrm{Pr}}(L_i)$.

The construction does not use all of $\mathsf{Occ}_{\mathrm{Pr}}(L_i)$ by default. Instead, the operational presentation carries a finite *site specification* $\mathfrak{S}$, a family

$$\mathfrak{S}_i \subseteq \mathsf{Occ}_{\mathrm{Pr}}(L_i)$$

choosing the source positions that are to generate contextual modal types. If a bare presentation is used without an explicit site specification, the default site policy in this paper is the finite family

$$\mathfrak{S}^{\mathrm{def}}_i = \mathsf{Occ}^{\mathrm{ch}}_{\mathrm{Pr}}(L_i).$$

Thus the default chooses proper constructor-headed process occurrences and excludes variable occurrences, structural s,t-occurrences, and the whole rewrite source. This default is only a convention for examples; a language designer may declare a smaller or larger finite set. In particular, variable occurrences and the whole source occurrence generate modalities only when they are explicitly declared. The separate possibility type $\Diamond$ below covers the identity-hole case, so the whole source is not included by default.

For example, in the π-calculus, the comm rule says

$$n, m : \mathrm{Nm}, k : \mathrm{Nm} \to \mathrm{Pr} \vdash \mathrm{comm} : n!m \mid n?k \rightsquigarrow k(m).$$

The proper constructor-headed source occurrences with carrier Pr are $n!m$ and $n?k$. The default site policy declares both. The input occurrence $n?k$, with displayed context $K[-] = n!m \mid [-]$, gives the receive modality

$$\langle n!m \mid [-] \rangle^{n::B}_{m::A} C.$$

The output occurrence has the dual displayed context $K[-] = [-] \mid n?k$, but its interface is not the simple expression $k :: A$: it involves the higher-order continuation parameter $k : \mathrm{Nm} \to \mathrm{Pr}$ and a different ambient/rely/index split from the receive site. We therefore leave the dual modality in schematic site notation unless an explicit type discipline for continuations has been chosen. These are declarations about the two children of the displayed source tree $n!m \mid n?k$. The commutativity equation for $\mid$ does not collapse them into one site and does not add the reversed-site contexts $n?k \mid [-]$ or $[-] \mid n!m$. Declaring only the input occurrence gives the receive fragment used later; declaring neither adds no communication-specific contextual modality.

Definition A.1 (Site specification and process sites)**.** *A site-marked operational presentation is a pair $(\mathcal{P}, \mathfrak{S})$ where $\mathcal{P}$ is a finite operational presentation and $\mathfrak{S}$ is a finite site specification as above. Its process-site set is*

$$\mathsf{Sites}(\mathcal{P}, \mathfrak{S}) = \{(i, o) \mid o \in \mathfrak{S}_i\}.$$

When the site specification is fixed, we write $\mathsf{Sites}(\mathcal{P})$ for $\mathsf{Sites}(\mathcal{P}, \mathfrak{S})$. For $s = (i, o) \in \mathsf{Sites}(\mathcal{P})$, write

$$U_s : \mathrm{Pr}$$

for the selected displayed subterm occurrence. There is a displayed one-hole process context $K_s[-]$ such that

$$L_i = K_s[U_s].$$

This is equality of displayed syntax trees, not equality modulo the equations of the operational theory. The pair (U_s, K_s) is the modal site generated by that declared occurrence. Two sites are equal only when their primitive rewrite constructors and occurrence addresses agree.

Definition A.2 (Closure support for a site)**.** *A declared site records a displayed redex context; it does not say that arbitrary rewrites may be propagated through that context. A* closure support *for a site s is a chosen displayed contextual-closure constructor κ_s in the operational presentation which transports a rewrite witness in the hole through the displayed site context. Schematically, for the same ambient, index, and rely parameters as the site, it has the form*

$$r : P \rightsquigarrow Q \quad \longmapsto \quad \kappa_s(r) : K_s[P] \rightsquigarrow K_s[Q].$$

After substituting ambient, index, and rely arguments, the same displayed constructor therefore gives

$$\kappa_s(r) : K_s[P][\vec{c}/\vec{z}, \vec{a}/\vec{x}, \vec{b}/\vec{y}] \rightsquigarrow K_s[Q][\vec{c}/\vec{z}, \vec{a}/\vec{x}, \vec{b}/\vec{y}].$$

We write $\kappa_s \Vdash K_s[-]$ when such a closure support has been specified, and call s closure-supported*. Closure support is separate from site declaration: it is not inferred from a judgment $P :: \Diamond C$, from the fact that $K_s[-]$ is a syntactic one-hole context, or from structural equations. It must be supplied by an explicit operational rule, such as a head context rule for lambda calculus or a parallel-context rule for process calculus.*

Because $\mathcal{P}$ and $\mathfrak{S}$ are finite, $\mathsf{Sites}(\mathcal{P})$ is finite. The site specification also fixes the intended granularity of the modal completion. For the lambda beta rule, the product completion declares the function-position occurrence and not the argument occurrence. For the asynchronous communication rule, the receive completion declares the input occurrence and not necessarily the output occurrence.

The free variables of the selected subterm and its surrounding context determine the modal interface, together with an ambient telescope:

$$\vec{x}_s = \mathsf{FV}(K_s) \cap \mathsf{FV}(U_s),$$

$$\vec{y}_s = \mathsf{FV}(K_s) \setminus \mathsf{FV}(U_s),$$
$$\vec{z}_s = \big(\mathsf{FV}(U_s) \setminus \mathsf{FV}(K_s)\big) \cup \big(\mathsf{FV}(R_i) \setminus \mathsf{FV}(L_i)\big).$$

The variables $\vec{x}_s$ are the *index variables*; they occur both in the term being typed and in the surrounding context, and they remain in the external context of the modal type. The variables $\vec{y}_s$ are the *rely variables*; they are supplied by the surrounding context and discharged by the modal type former. The variables $\vec{z}_s$ are the *ambient variables*; they are ordinary parameters of the selected subterm or of the target which are not supplied by the one-hole context and are not discharged by the modal type former.

For each site, let Θ_s be the finite telescope of declarations for $\vec{z}_s$ inherited from the primitive rewrite context Γ_i, Δ_i. The ambient telescope is not part of the modal interface and contributes no profile slots. It is a side-context for the generated rules: when the rules below write Γ, Θ_s, they mean an arbitrary surrounding context Γ extended by declarations making all variables in $\vec{z}_s$ well formed. If $\vec{z}_s$ is empty, Θ_s is omitted.

A.4 Consistency relations on profiles

For a site s, write

$$I_s = \{i_x \mid x \in \vec{x}_s\} \amalg \{r_y \mid y \in \vec{y}_s\} \amalg \{\tau_s\}.$$

The slot i_x records the sort chosen for the type of the index variable x, the slot r_y records the sort chosen for the type of the rely variable y, and τ_s records the sort chosen for the target type C. Ambient variables have no slots in I_s: their declarations are ordinary assumptions in Θ_s, not type-forming axes of the contextual modality. If there are m_s index variables and k_s rely variables, then I_s has $m_s + k_s + 1$ elements.

We will use an explicit finite labelling of the type occurrences whose sorts are controlled by these slots. Let $\mathsf{TOcc}_{\mathcal{P},s}$ be the finite set of *profile-relevant type occurrences* appearing in the displayed rule templates generated from the site s, together with the profile-relevant type occurrences in displayed equations that are explicitly matched to such rule occurrences. There is a total slot-labelling map

$$\ell_s : \mathsf{TOcc}_{\mathcal{P},s} \longrightarrow I_s.$$

For example, an occurrence of the type attached to an index or rely variable v is labelled by the corresponding slot e_v, and an occurrence of the result type of the primitive rewrite target is labelled by the target slot τ_s. We also write

$$\mu_s : \mathsf{TOcc}_{\mathcal{P},s} \rightharpoonup \mathsf{TMeta}$$

for the partial map that records the displayed type metavariable, when an occurrence is literally named by one.

For each displayed equation $E : L = R$ and site s, let

$$\mathsf{Match}_s(E) \subseteq \mathsf{TOcc}_{\mathcal{P},s} \times \mathsf{TOcc}_{\mathcal{P},s}$$

be the finite relation of profile-relevant type occurrences that the equation schema matches for profile purposes. This relation is not the structural congruence generated by E; it is extra finite bookkeeping attached to the displayed equation. When no matching data are written, we use the following syntactic default. The default relation contains exactly those pairs (o, o') of occurrences on opposite sides of E such that either $\mu_s(o) = \mu_s(o')$ is defined, or o and o' occur at the same tree address inside subterms with identical constructor heads and arities. In both cases the

carriers must agree. Structural equations that move occurrences to different addresses, such as commutativity or associativity, therefore contribute no such moved identifications unless their matching pairs are explicitly listed in $\mathsf{Match}_s(E)$.

The construction of the profile quotient is a finite syntactic preprocessing step. It uses the displayed finite presentation $\mathcal{P}$, including the finite relations $\mathsf{Match}_s(E)$, not the full congruence or equational theory generated by $\mathcal{P}$. Thus a listed equation contributes constraints only through its displayed finite occurrence matching. The same equation still belongs to the equational layer and may be used for ordinary conversion in the generated theory; that conversion role is separate from its profile-matching role. We do not enumerate consequences obtained by substitution into arbitrary contexts, by transitivity of equality, or by rewriting modulo the listed equations.

Each slot $e \in I_s$ has a carrier, written $\mathrm{car}(e)$: $\mathrm{car}(i_x)$ is the carrier of x, $\mathrm{car}(r_y)$ is the carrier of y, and $\mathrm{car}(\tau_s) = \mathrm{Pr}$. The slot attached to a variable of carrier X ranges over the two generated sorts $\{*^X, \square^X\}$, and the target slot ranges over $\{*^{\mathrm{Pr}}, \square^{\mathrm{Pr}}\}$. When all slots have carrier Pr, we suppress the carrier superscripts and write the raw profile set as

$$\{*, \square\}^{I_s} \cong \{*, \square\}^{m_s+k_s+1}.$$

In general, the raw profile set is

$$\mathsf{RawPrfl}_{\mathcal{P}}(s) = \prod_{e \in I_s} \{*^{\mathrm{car}(e)}, \square^{\mathrm{car}(e)}\}.$$

Definition A.3 (Profile-consistency algorithm)**.** *For each site s, form a finite undirected graph $G_{\mathcal{P},s}$ with vertex set I_s, initialized with no edges. The following finite clauses propose edges. A proposed edge is inserted exactly when it passes the carrier check in clause* (v).

(i) ***Shared type metavariable.*** *For occurrences $o, o' \in \mathsf{TOcc}_{\mathcal{P},s}$, if $\mu_s(o)$ and $\mu_s(o')$ are defined and equal, propose the edge $\ell_s(o) \sim \ell_s(o')$. For example, if the same displayed type metavariable is used both as a binder annotation and as the domain of the generated modal type, the annotation slot and the modal-domain slot are identified.*

(ii) ***Binder annotation.*** *For every Church-style refinement of a binding constructor, let o_A be the type occurrence of the added annotation and let o_x be the occurrence of the type of the variable bound by that constructor. Propose the edge $\ell_s(o_A) \sim \ell_s(o_x)$. This clause is applied only to the finitely many binding-constructor occurrences appearing in the displayed source, selected subterm, surrounding context, and target of the primitive rewrite constructor under consideration.*

(iii) ***Target variable.*** *Suppose the target R_i of the primitive rewrite constructor defining the site is a displayed occurrence of a variable $v \in \vec{x}_s \cup \vec{y}_s$. Let e_v be the index or rely slot attached to the declaration of v, and let τ_s be the target slot. Propose the edge $e_v \sim \tau_s$. This records the fact that the target-typing premise for R_i uses the contextual declaration of the same variable. The clause deliberately ranges only over index and rely variables: ambient variables in Θ_s are external parameters and do not have profile slots.*

(iv) ***Listed equation.*** *For each displayed equation E of $\mathcal{P}$ and each matched pair $(o, o') \in \mathsf{Match}_s(E)$, propose the edge $\ell_s(o) \sim \ell_s(o')$. This is the only way listed equations constrain profile slots: the algorithm does not derive additional slot identifications from the equational theory generated by the presentation.*

(v) ***Carrier check.*** *A proposed edge $e \sim e'$ is inserted only if* $\mathrm{car}(e) = \mathrm{car}(e')$*, unless the presentation explicitly contains a displayed carrier identification making the comparison well sorted. In the ordinary case, this means that* Pr*-slots are compared only with* Pr*-slots and* Nm*-slots only with* Nm*-slots.*

Let $\mathsf{Con}_{\mathcal{P},s}$ be the reflexive, symmetric, transitive closure of the inserted finite edge relation. Equivalently, $\mathsf{Con}_{\mathcal{P},s}$ is computed by a union-find pass over the finite graph $G_{\mathcal{P},s}$.

The quotient profile space is the set of raw profiles that are constant on the $\mathsf{Con}_{\mathcal{P},s}$-classes:

$$\mathsf{Prfl}_{\mathcal{P}}(s) = \left\{\omega \in \mathsf{RawPrfl}_{\mathcal{P}}(s) \;\middle|\; e\, \mathsf{Con}_{\mathcal{P},s}\, e' \Rightarrow \omega(e) = \omega(e')\right\}.$$

Equivalently,

$$\mathsf{Prfl}_{\mathcal{P}}(s) \cong \prod_{[e] \in I_s/\mathsf{Con}_{\mathcal{P},s}} \{*^{\mathrm{car}(e)}, \square^{\mathrm{car}(e)}\},$$

where the carrier check makes $\mathrm{car}(e)$ well defined on each equivalence class. Thus the algorithm for computing $\mathsf{Prfl}_{\mathcal{P}}(s)$ is:

(1) extract the finite site interface I_s, the occurrence set $\mathsf{TOcc}_{\mathcal{P},s}$, the slot-labelling map ℓ_s, and the displayed equation matchings $\mathsf{Match}_s(E)$,

(2) add the finite set of carrier-correct edges proposed above,

(3) compute the equivalence classes of $G_{\mathcal{P},s}$, and

(4) enumerate one independent $*/\square$ choice for each equivalence class.

Definition A.4 (Signature choice)**.** *A signature choice for a site-marked finite presentation $(\mathcal{P}, \mathfrak{S})$ is a family of subsets*

$$\Sigma_s \subseteq \mathsf{Prfl}_{\mathcal{P}}(s) \qquad (s \in \mathsf{Sites}(\mathcal{P})).$$

Equivalently, it is a subset

$$\Sigma \subseteq \coprod_{s \in \mathsf{Sites}(\mathcal{P})} \mathsf{Prfl}_{\mathcal{P}}(s).$$

The chosen typing discipline is the subset Σ of the quotient profile space.

Remark A.5 (Structural equations do not merge sites)**.** *For the asynchronous communication source $n!m \mid n?k$, the site specification may name the left output occurrence, the right input occurrence, both, or neither. These names are occurrences in the displayed source tree. The structural equation $p \mid q = q \mid p$ identifies the two process terms $n!m \mid n?k$ and $n?k \mid n!m$ in the equational layer, but it does not identify the two declared sites, create a reversed-context site, or add a profile edge. Such effects occur only when the site specification or the profile-matching data explicitly says so.*

Thus the input is not an arbitrary family of contexts and not an arbitrary family of sort tuples. The finite presentation together with its site specification determines the displayed sites, and the separate equation-matching data determines the quotient profile spaces. The user, or language designer, chooses which operational occurrences are sites, decides which structural equations, if any, should carry profile matchings, may annotate selected sites with explicit contextual-closure

constructors for delayed elimination, and then chooses a subset of the quotient profile space at each declared site.

If $\mathsf{Prfl}_{\mathcal{P}}(s)$ has N elements, then the possible subsets Σ_s form an N-dimensional Boolean hypercube. Requiring some base profiles cuts out a face of that hypercube. This is the general version of the Lambda cube: different subsets of the available quotient profiles determine different corners or faces of a hypercube of modal dependent type systems.

Example A.6 (Lambda profiles)**.** *For the beta source*

$$\mathsf{app}(\mathsf{lam}(t), y) \rightsquigarrow t(y),$$

the product-site specification declares the function-position subterm $U = \mathsf{lam}(t)$. This is also the only occurrence selected by the default site policy, since the argument y is a variable occurrence and the whole source is excluded. The declared site gives the context

$$K[-] = \mathsf{app}([-], y).$$

There is one rely variable, y, and no index variable. Thus the raw profile set is

$$\{*, \Box\}^2.$$

With no index slot, a profile is written $(\beta; \gamma)$, where β is the sort of the argument type and γ is the sort of the result type. If the presentation imposes no further consistency relation between the domain slot and target slot, the quotient profile space consists of

$$(*; *), \qquad (*; \Box), \qquad (\Box; *), \qquad (\Box; \Box).$$

Choosing only $(*; *)$ gives the simply typed corner. Choosing all four gives the CoC corner for the product site.

Example A.7 (A nontrivial profile quotient)**.** *Consider a toy operational presentation with constructors*

$$\mathsf{mark} : \mathrm{Pr} \to \mathrm{Pr}, \qquad \mathsf{choose} : \mathrm{Pr} \times \mathrm{Pr} \times \mathrm{Pr} \to \mathrm{Pr},$$

and primitive rewrite constructor

$$x, y : \mathrm{Pr} \vdash \rho : \mathsf{choose}(\mathsf{mark}(x), y, x) \rightsquigarrow y.$$

Declare the site

$$U = \mathsf{mark}(x), \qquad K[-] = \mathsf{choose}([-], y, x).$$

Then x is an index variable, since it occurs both in U and in $K[-]$, and y is a rely variable, since it occurs in $K[-]$ but not in U. The raw slots are

$$i_x, \qquad r_y, \qquad \tau,$$

for the type of x, the type of y, and the target type. Before imposing consistency, the raw profiles are triples $(\alpha, \beta; \gamma) \in \{*, \Box\}^3$, where α is the index sort and β is the rely sort.

The target of the rewrite is the displayed variable y. Clause (iii) *of the profile-consistency algorithm therefore adds the edge $r_y \sim \tau$. No displayed constraint mentions i_x. Hence the quotient has two independent classes,*

$$\{i_x\}, \qquad \{r_y, \tau\},$$

and the profile space is

$$\mathsf{Prfl}_{\mathcal{P}}(s) = \{(\alpha, \beta; \beta) \mid \alpha, \beta \in \{*, \Box\}\}.$$

Thus this site has four profiles rather than the eight profiles of the raw three-slot cube.

B Generated presentations and structured categories

This appendix records the full generated-presentation data summarized in Section 5.

We separate the generated typed presentation from the categorical object used for the universal property. If

$$A = (\mathcal{P}, \mathfrak{S}, \Sigma)$$

is a site-marked operational presentation with a signature choice, write $\mathcal{G}(A)$ for the finite typed presentation obtained by the hypercube construction. Outside categorical statements we also write $\mathsf{Hyp}(A)$ informally for this generated presentation; categorically, $\mathsf{Hyp}(A)$ will be the corresponding free structured object defined below.

The source category. Let **PSig** be the category whose objects are triples

$$(\mathcal{P}, \mathfrak{S}, \Sigma),$$

where $\mathcal{P}$ is a finite presentation of an operational theory, $\mathfrak{S}$ is a finite site specification, and Σ is a signature choice, i.e. a family of subsets

$$\Sigma_s \subseteq \mathsf{Prfl}_{\mathcal{P}}(s) \qquad (s \in \mathsf{Sites}(\mathcal{P}, \mathfrak{S})).$$

A morphism

$$F : (\mathcal{P}, \mathfrak{S}, \Sigma) \longrightarrow (\mathcal{P}', \mathfrak{S}', \Sigma')$$

in **PSig** is a strict presentation morphism sending carrier objects, constructors, relations, equations, profile-matching annotations, closure-support annotations, and primitive rewrite constructors of $\mathcal{P}$ to corresponding displayed generators of $\mathcal{P}'$, preserving Pr, R, and s, t. Strictness means that, for each primitive rewrite constructor $\rho_i : L_i \rightsquigarrow R_i$, F sends the displayed source tree L_i to the displayed source tree of $F(\rho_i)$ with occurrence addresses preserved, before quotienting by structural equations. Thus if $o \in \mathfrak{S}_i$, then the image occurrence $F(o)$ lies in $\mathfrak{S}'_{F(i)}$, and F induces

$$\mathsf{Sites}(F) : \mathsf{Sites}(\mathcal{P}, \mathfrak{S}) \longrightarrow \mathsf{Sites}(\mathcal{P}', \mathfrak{S}').$$

It also preserves matching data and closure support:

$$(o, o') \in \mathsf{Match}_s(E) \quad \Longrightarrow \quad (F(o), F(o')) \in \mathsf{Match}_{\mathsf{Sites}(F)(s)}(F(E)),$$

and $\kappa_s \Vdash K_s[-]$ is sent to $F(\kappa_s) \Vdash K_{\mathsf{Sites}(F)(s)}[-]$. Hence F induces maps of quotient profile spaces

$$\mathsf{Prfl}_{\mathcal{P}}(s) \longrightarrow \mathsf{Prfl}_{\mathcal{P}'}(\mathsf{Sites}(F)(s)).$$

Finally, F is signature-preserving:

$$\omega \in \Sigma_s \quad \Longrightarrow \quad F(\omega) \in \Sigma'_{\mathsf{Sites}(F)(s)}.$$

Identities and composition are inherited from strict presentation morphisms; the displayed-address, matching, closure-support, and signature-preservation conditions are stable under composition.

The generated typed presentation. On an object $A = (\mathcal{P}, \mathfrak{S}, \Sigma)$, the presentation $\mathcal{G}(A)$ is obtained by adjoining:

(i) the two sort constants $*^X, \square^X$ for each relevant carrier X,

(ii) the sort axiom $*^X ::^X \square^X$,

(iii) proof-relevant typing evidence displays $p_X : \mathsf{Ev}_X \to X \times X$,

(iv) Church-style annotations for binding constructors,

(v) the one-step possibility type $\Diamond C$,

(vi) for every site s and every chosen quotient profile in Σ_s, a contextual modal type former,

(vii) the source-subterm, modal introduction, modal elimination, and computation rules generated from these formers,

(viii) and no primitive delayed-$\Diamond$ rules. Delayed elimination for closure-supported sites is an admissible metatheorem, not part of the finite generated presentation.

On a morphism $F : A \to A'$, the induced map $\mathcal{G}(F)$ acts by applying F to old syntax and rewrites and by sending each generated component at a site s and profile $\omega \in \Sigma_s$ to the generated component at $\mathsf{Sites}(F)(s)$ and the image profile $F(\omega) \in \Sigma'_{\mathsf{Sites}(F)(s)}$. The functoriality of $\mathcal{G}$ is proved in Appendix C.

The target category. Let **DOpPres** be the category of finite dependently typed operational presentations and strict morphisms of such presentations. A strict morphism preserves the named carrier objects, constructors, relations, equations, binding structure, typing judgments, and inference-rule schemas; equivalently, it sends every rule of the source presentation to the corresponding named rule, or to a specified derivation of that rule, in the target presentation.

The target of the universal property is not the unstructured category **DOpPres**, but the category of typed operational presentations equipped with an interpretation of the generated hypercube structure. Define **PDTOT** as follows. An object is a triple

$$X = (A, \mathcal{T}, \alpha_X)$$

where $A \in \mathbf{PSig}$, $\mathcal{T} \in \mathbf{DOpPres}$, and

$$\alpha_X : \mathcal{G}(A) \longrightarrow \mathcal{T}$$

is a strict morphism of typed presentations. Thus α_X chooses, in $\mathcal{T}$, interpretations of the generated sort constants, typing evidence displays, Church annotations, possibility type, contextual modalities, and rules belonging to A. The presentation $\mathcal{T}$ may contain additional generators, equations, or rules; freeness only concerns the named structure selected by α_X.

A morphism

$$(F, h) : (A, \mathcal{T}, \alpha_X) \longrightarrow (B, \mathcal{T}', \alpha_Y)$$

in **PDTOT** consists of a morphism $F : A \to B$ in **PSig** and a strict morphism

$$h : \mathcal{T} \to \mathcal{T}'$$

in **DOpPres** such that

$$h \circ \alpha_X = \alpha_Y \circ \mathcal{G}(F).$$

Equivalently, the square

$$\begin{array}{ccc} \mathcal{G}(A) & \xrightarrow{\mathcal{G}(F)} & \mathcal{G}(B) \\ {\scriptstyle \alpha_X}\downarrow & & \downarrow{\scriptstyle \alpha_Y} \\ \mathcal{T} & \xrightarrow{h} & \mathcal{T}' \end{array}$$

commutes. Identities and composition are componentwise. The forgetful functor

$$U : \mathbf{PDTOT} \longrightarrow \mathbf{PSig}$$

is defined by

$$U(A, \mathcal{T}, \alpha_X) = A, \qquad U(F, h) = F.$$

The hypercube functor is the free structured extension

$$\mathsf{Hyp} : \mathbf{PSig} \longrightarrow \mathbf{PDTOT}, \qquad \mathsf{Hyp}(A) = (A, \mathcal{G}(A), \mathrm{id}_{\mathcal{G}(A)}),$$

and on morphisms

$$\mathsf{Hyp}(F) = (F, \mathcal{G}(F)).$$

Theorem 14.1 states the adjunction in the main text, and Appendix C gives the full proof.

Proposition B.1 (Finiteness)**.** *If $\mathcal{P}$ is finite, $\mathfrak{S}$ is a finite site specification, and Σ is a signature choice for $(\mathcal{P}, \mathfrak{S})$, then the underlying generated presentation $\mathcal{G}(\mathcal{P}, \mathfrak{S}, \Sigma)$, equivalently $\mathsf{Hyp}(\mathcal{P}, \mathfrak{S}, \Sigma)$ outside the categorical section, is finitely generated.*

Proof. There are finitely many primitive rewrite constructors, each raw displayed source term has finitely many Pr-subterm occurrences, and $\mathfrak{S}$ chooses only finitely many of them. Hence $\mathsf{Sites}(\mathcal{P})$ is finite. The free-variable lists $\vec{x}_s$, $\vec{y}_s$, and $\vec{z}_s$, and hence the ambient telescope Θ_s, are finite for each site. Each displayed equation has only finitely many explicit or default profile matchings. For each site, the raw profile set $\{*, \square\}^{m_s+k_s+1}$ is finite, and its consistency quotient is finite. Therefore the chosen subset Σ_s is finite. The construction adds finitely many sort constants, typing evidence displays, Church-style refinements, modal formers, and rule schemas generated from this finite data. □

C Full proof of functoriality and adjunction

This appendix supplies the details behind Theorem 14.1.

Let

$$F : A = (\mathcal{P}, \mathfrak{S}, \Sigma) \longrightarrow A' = (\mathcal{P}', \mathfrak{S}', \Sigma')$$

be a morphism in **PSig**. The generated-presentation map

$$\mathcal{G}(F) : \mathcal{G}(A) \longrightarrow \mathcal{G}(A')$$

is defined by structural recursion on generators and rules. On old raw syntax, primitive rewrites, equations, site declarations, profile matchings, and closure-support annotations, $\mathcal{G}(F)$ is F. On generated sorts and typing evidence displays it sends

$$*^X \mapsto *^{F(X)}, \qquad \square^X \mapsto \square^{F(X)}, \qquad (M ::^X A_0) \mapsto (F(M) ::^{F(X)} F(A_0)).$$

On a generated modality at site s and profile $\omega \in \Sigma_s$, it sends

$$\langle K_s \rangle^{\vec{x}::\vec{A}}_{\vec{y}::\vec{B}} C$$

to

$$\left\langle K_{\mathsf{Sites}(F)(s)} \right\rangle^{F(\vec{x})::F(\vec{A})}_{F(\vec{y})::F(\vec{B})} F(C).$$

This target former exists because F preserves declared sites and chosen profiles. The action on inference rules is obtained by applying F to every premise and conclusion, including ambient telescopes and closure supports. For example, modal introduction is sent to modal introduction at the image site:

$$\frac{\begin{array}{r} F(\Gamma), F(\vec{x}) : F(\vec{X}_s), F(\vec{y}) : F(\vec{Y}_s) \mid F(\Delta), F(\vec{e}_x) : F(\vec{x}) :: F(\vec{A}), F(\vec{e}_y) : F(\vec{y}) :: F(\vec{B}) \vdash F(r) : K_{\mathsf{Sites}(F)(s)}[F(P)] \rightsquigarrow \\ F(\Gamma), F(\vec{x}) : F(\vec{X}_s), F(\vec{y}) : F(\vec{Y}_s) \mid F(\Delta), F(\vec{e}_x) : F(\vec{x}) :: F(\vec{A}), F(\vec{e}_y) : F(\vec{y}) :: F(\vec{B}) \vdash F(Q) :: F(C) \end{array}}{F(\Gamma), F(\vec{x}) : F(\vec{X}_s) \mid F(\Delta), F(\vec{e}_x) : F(\vec{x}) :: F(\vec{A}) \vdash F(P) :: \left\langle K_{\mathsf{Sites}(F)(s)} \right\rangle^{F(\vec{x})::F(\vec{A})}_{F(\vec{y})::F(\vec{B})} F(C)}$$

The same clause handles source-subterm introduction, modal elimination, and computation equations. Delayed elimination is not a generated rule to be mapped; closure-support preservation instead ensures that the admissibility theorem for source sites transports to the corresponding admissibility theorem for target sites.

Proposition C.1 (Functoriality of the generated presentation)**.** *The assignment*

$$\mathcal{G} : \mathbf{PSig} \longrightarrow \mathbf{DOpPres}$$

is a functor.

Proof. For each $F : A \to B$ in **PSig**, the preceding structural-recursion clauses define a strict morphism $\mathcal{G}(F) : \mathcal{G}(A) \to \mathcal{G}(B)$. The only nontrivial well-definedness checks are the generated components indexed by sites, profiles, ambient telescopes, and closure supports. These are precisely the preservation requirements in the definition of **PSig**-morphism: displayed addresses send sites to sites; matching annotations send consistent profiles to consistent profiles; signature preservation sends chosen profiles to chosen profiles; and closure-support preservation sends the hypotheses needed for admissible delayed elimination to the corresponding target hypotheses. Therefore every generated symbol, equation, and primitive rule of $\mathcal{G}(A)$ has a named image in $\mathcal{G}(B)$.

If $F = \mathrm{id}_A$, the recursive clauses send every old and generated generator to itself, hence $\mathcal{G}(F) = \mathrm{id}_{\mathcal{G}(A)}$. If $F : A \to B$ and $G : B \to C$, then $\mathcal{G}(G) \circ \mathcal{G}(F)$ and $\mathcal{G}(G \circ F)$ agree on old generators by functoriality of strict presentation morphisms, and they agree on generated generators because all clauses are defined by applying the underlying map to the relevant carrier, site, profile, telescope, and closure-support data. Since $\mathcal{G}(A)$ is generated by exactly these old and generated symbols and rules, the two morphisms are equal. □

Proposition C.2 (The structured target category)**.** *The data in Appendix B define a category* **PDTOT***, and*

$$U : \mathbf{PDTOT} \longrightarrow \mathbf{PSig}$$

is a functor.

Proof. For an object $X = (A, \mathcal{T}, \alpha_X)$, the identity morphism is

$$(\mathrm{id}_A, \mathrm{id}_{\mathcal{T}}).$$

The defining square commutes because

$$\mathrm{id}_{\mathcal{T}} \circ \alpha_X = \alpha_X \circ \mathcal{G}(\mathrm{id}_A).$$

If

$$(F, h) : X \to Y, \qquad (F', h') : Y \to Z,$$

then their composite is $(F' \circ F, h' \circ h)$. If $Y = (B, \mathcal{T}', \alpha_Y)$ and $Z = (C, \mathcal{T}'', \alpha_Z)$, the square for the composite commutes because

$$h' \circ h \circ \alpha_X = h' \circ \alpha_Y \circ \mathcal{G}(F) = \alpha_Z \circ \mathcal{G}(F') \circ \mathcal{G}(F) = \alpha_Z \circ \mathcal{G}(F' \circ F).$$

Associativity and identity laws are inherited componentwise from **PSig** and **DOpPres**. The functor U keeps only the first component, so it preserves identities and composition. □

Theorem C.3 (Free hypercube completion)**.** *The functor*

$$\mathsf{Hyp} : \mathbf{PSig} \longrightarrow \mathbf{PDTOT}$$

is left adjoint to the forgetful functor

$$U : \mathbf{PDTOT} \longrightarrow \mathbf{PSig}.$$

Equivalently, for every $A \in \mathbf{PSig}$ and every $X = (B, \mathcal{T}, \alpha_X) \in \mathbf{PDTOT}$, there is a natural bijection

$$\mathrm{Hom}_{\mathbf{PDTOT}}(\mathsf{Hyp}(A), X) \cong \mathrm{Hom}_{\mathbf{PSig}}(A, U(X)).$$

Proof. First, Hyp is a functor. By definition,

$$\mathsf{Hyp}(A) = (A, \mathcal{G}(A), \mathrm{id}_{\mathcal{G}(A)}), \qquad \mathsf{Hyp}(F) = (F, \mathcal{G}(F)).$$

The square required for $\mathsf{Hyp}(F)$ to be a **PDTOT**-morphism is

$$\begin{array}{ccc} \mathcal{G}(A) & \xrightarrow{\mathcal{G}(F)} & \mathcal{G}(B) \\ {\scriptstyle \mathrm{id}}\downarrow & & \downarrow{\scriptstyle \mathrm{id}} \\ \mathcal{G}(A) & \xrightarrow{\mathcal{G}(F)} & \mathcal{G}(B), \end{array}$$

which commutes. Identities and composition follow from the functoriality of $\mathcal{G}$.

Now fix $A \in \mathbf{PSig}$ and $X = (B, \mathcal{T}, \alpha_X) \in \mathbf{PDTOT}$. A morphism

$$(F, h) : \mathsf{Hyp}(A) \to X$$

in **PDTOT** consists of a morphism $F : A \to B$ in **PSig** and a strict morphism $h : \mathcal{G}(A) \to \mathcal{T}$ in **DOpPres** satisfying

$$h \circ \mathrm{id}_{\mathcal{G}(A)} = \alpha_X \circ \mathcal{G}(F).$$

Thus h is forced to be $\alpha_X \circ \mathcal{G}(F)$. Consequently the map

$$(F, h) \longmapsto F$$

from $\mathrm{Hom}_{\mathbf{PDTOT}}(\mathsf{Hyp}(A), X)$ to $\mathrm{Hom}_{\mathbf{PSig}}(A, U(X))$ is injective. It is also surjective: given $F : A \to U(X) = B$, the pair

$$\big(F, \alpha_X \circ \mathcal{G}(F)\big)$$

is a **PDTOT**-morphism $\mathsf{Hyp}(A) \to X$ because it satisfies the commuting-square condition by construction. Hence the displayed hom-set map is a bijection.

Naturality is componentwise. Precomposition by a morphism $A' \to A$ in **PSig** replaces F by its composite on the left, and the corresponding **DOpPres**-component is forced to be the same composite under $\mathcal{G}$. Postcomposition by a morphism $(G, k) : X \to Y$ in **PDTOT** replaces F by $G \circ F$; the commuting-square condition for (G, k) gives

$$k \circ \alpha_X \circ \mathcal{G}(F) = \alpha_Y \circ \mathcal{G}(G) \circ \mathcal{G}(F) = \alpha_Y \circ \mathcal{G}(G \circ F),$$

which is exactly the forced second component for the composite. Therefore the bijection is natural in both variables.

The unit at A is the identity morphism

$$\eta_A = \mathrm{id}_A : A \longrightarrow U\mathsf{Hyp}(A) = A,$$

and the counit at $X = (B, \mathcal{T}, \alpha_X)$ is

$$\epsilon_X = (\mathrm{id}_B, \alpha_X) : \mathsf{Hyp}(U(X)) = (B, \mathcal{G}(B), \mathrm{id}_{\mathcal{G}(B)}) \longrightarrow (B, \mathcal{T}, \alpha_X).$$

The first triangle identity is

$$\epsilon_{\mathsf{Hyp}(A)} \circ \mathsf{Hyp}(\eta_A) = (\mathrm{id}_A, \mathrm{id}_{\mathcal{G}(A)}),$$

and the second is

$$U(\epsilon_X) \circ \eta_{U(X)} = \mathrm{id}_{U(X)}.$$

Therefore $\mathsf{Hyp} \dashv U$. □

This theorem is the formal meaning of "freely closes an operational theory under the generated type-forming structure": to interpret the free completion in any structured typed operational theory X, it is necessary and sufficient to give a strict morphism of the underlying site-and-signature data into $U(X)$. The remaining generated sorts, type formers, and rules are then forced by the structure map α_X.

D Full proof-relevant evidence semantics and soundness proof

This appendix supplies the proof details behind Theorem 15.1 and Proposition 15.2.

The semantic statement has two layers. The primary layer is proof-relevant: typing judgments denote evidence objects, and modal judgments retain the rewrite witnesses that justify them. The optional second layer forgets that evidence and forms proof-irrelevant may-predicates by taking images and dependent universal operations.

Let $\mathcal{E}$ be a category with the finite limits and specified exponentials needed to interpret the underlying lambda-theory presentation. A proof-relevant model of the generated typed presentation $\mathcal{H}(A)$ in $\mathcal{E}$ interprets each carrier X as an object, each constructor as a morphism, and each equation as an equality of morphisms. For each carrier X, the typing judgment is interpreted by a display

$$p_X : \mathsf{Ev}_X \longrightarrow X \times X, \qquad p_X = \langle \mathsf{tm}_X, \mathsf{tp}_X \rangle.$$

The map p_X is not required to be monic. The fiber over (p, a) is the object of evidence witnessing $p ::^X a$.

If a type term $A : X$ is interpreted in context $\Gamma \mid \Delta$ by a morphism

$$[\![A]\!]_{\Gamma|\Delta} : [\![\Gamma \mid \Delta]\!] \longrightarrow X,$$

then typed context extension is the pullback

$$[\![\Gamma, x : X \mid \Delta, e_x : x ::^X A]\!] = ([\![\Gamma \mid \Delta]\!] \times X) \times_{X \times X} \mathsf{Ev}_X,$$

where the comparison map $[\![\Gamma \mid \Delta]\!] \times X \to X \times X$ is $\langle \pi_2, [\![A]\!]_{\Gamma|\Delta}\pi_1 \rangle$. Thus a point of $[\![\Gamma, x : X \mid \Delta, e_x : x ::^X A]\!]$ is a triple (c, x, e_x), where $c \in [\![\Gamma \mid \Delta]\!]$ and e_x is evidence that x has type $A(c)$. Weakening, exchange, and substitution are interpreted by the usual pullback maps between these evidence contexts.

A typing judgment

$$\Gamma \mid \Delta \vdash P :: A$$

is valid when there is an evidence morphism

$$e_P : [\![\Gamma \mid \Delta]\!] \longrightarrow \mathsf{Ev}_X$$

satisfying

$$p_X e_P = \left\langle [\![P]\!]_{\Gamma|\Delta}, [\![A]\!]_{\Gamma|\Delta} \right\rangle.$$

For non-typing atomic judgments, such as rewrite judgments, validity is interpreted by the corresponding evidence object supplied by the operational presentation. For example, a rewrite judgment $r : P \rightsquigarrow Q$ is interpreted by a morphism into R whose source and target projections are P and Q.

The possibility display. For a type term $C : \mathrm{Pr}$ in context $\Gamma \mid \Delta$, the proof-relevant evidence for $P :: \Diamond C$ is the finite-limit object of tuples

$$(Q, r, e_C) \quad \text{with} \quad r : P \rightsquigarrow Q \quad \text{and} \quad e_C : Q :: C.$$

Equivalently, the pullback of $p_{\mathrm{Pr}} : \mathsf{Ev}_{\mathrm{Pr}} \to \mathrm{Pr} \times \mathrm{Pr}$ along $(P, \Diamond C)$ is equipped, naturally in $\Gamma \mid \Delta$, with destructors to a reduct Q, a rewrite witness $r : P \rightsquigarrow Q$, and target evidence $e_C : Q :: C$, together with the corresponding introduction constructor. In the free generated model this display is exactly the witness object above; in an arbitrary structured model it is required to carry the same specified introduction, elimination, and computation structure.

The $\Diamond$-introduction rule is interpreted by the tuple constructor

$$(Q, r, e_C) \longmapsto \mathrm{diaIntro}(Q, r, e_C) \in \mathsf{Ev}_{\mathrm{Pr}}(P, \Diamond C).$$

The $\Diamond$-eliminator is interpreted by destructing an element of $\mathsf{Ev}_{\mathrm{Pr}}(P, \Diamond C)$ into Q, r, and e_C, and then applying the interpretation of the derivation in the extended context. The usual freshness condition says that the final conclusion is pulled back from the smaller context and does not mention the destructed variables.

Contextual modalities. For a declared site s, write

$$M_s = \langle K_s \rangle^{\vec{x}::\vec{A}}_{\vec{y}::\vec{B}} C.$$

Evidence for $P :: M_s$ is proof-relevant evidence for the site interface. The generated modal-elimination rule is interpreted by a morphism which, from

$$e_M : P :: M_s \qquad \text{and} \qquad \vec{e}_b : \vec{b} :: \vec{B},$$

produces evidence

$$\mathrm{elim}_s(e_M, \vec{b}, \vec{e}_b) : K_s[P][\vec{b}/\vec{y}] :: \Diamond\big(C[\vec{b}/\vec{y}]\big).$$

The generated modal-introduction rule supplies such an interface from open one-step evidence in the rely context:

$$r : K_s[P] \rightsquigarrow Q, \qquad e_C : Q :: C \quad \Longrightarrow \quad e_M : P :: M_s.$$

Semantically, this is the abstraction operation associated to the discharged rely telescope, using the specified binding/exponential structure of the model, or equivalently the named modal-introduction rule morphism of the structured presentation. No proof-irrelevant image is involved at this stage.

If $s = (i, o)$ and the displayed source factorization is $L_i = K_s[U_s]$, the source-subterm rule is the special case in which the one-step witness is the primitive rewrite constructor ρ_i. The computation equation says that source-subterm introduction followed by modal elimination gives the same $\Diamond$-evidence as direct $\Diamond$-introduction from ρ_i and the target typing evidence. Semantically this is an equality of morphisms into the possibility evidence display.

Theorem D.1 (Soundness of the generated rules)**.** *Let $A = (\mathcal{P}, \mathfrak{S}, \Sigma)$ be an object of* **PSig***, and let $\mathcal{M}$ be a structured proof-relevant model of the generated typed presentation $\mathcal{H}(A)$ in the sense above. If a judgment J is derivable from the generated rules of $\mathcal{H}(A)$, then $[\![J]\!]$ is valid in $\mathcal{M}$. In particular, if J is $\Gamma \mid \Delta \vdash P :: A$, then there is an evidence morphism*

$$e_P : [\![\Gamma \mid \Delta]\!] \longrightarrow \mathsf{Ev}_X$$

over $\langle [\![P]\!]_{\Gamma|\Delta}, [\![A]\!]_{\Gamma|\Delta} \rangle$.

Proof. The proof is by induction on the derivation of J. The structural rules are pullback facts. Weakening is pullback along the projection from the extended evidence context, and substitution is pullback along the section interpreting the substituted term together with its typing evidence. Equational conversion is sound because equations of the presentation are interpreted as equal morphisms, so evidence over one representative transports to evidence over the other by the specified conversion rule.

For each non-structural generated rule, a structured model contains exactly the morphism from the evidence object of its premises to the evidence object of its conclusion specified by that rule schema. The $\Diamond$-rules use the rewrite display $\mathrm{R} \rightrightarrows \mathrm{Pr}$ and the target typing evidence. The contextual modal rules use the displayed factorization $L_i = K_s[U_s]$, the index-first telescope, and the ambient telescope Θ_s. The source-subterm rule is sound because the substituted primitive rewrite constructor ρ_i is a point of the modal-introduction premise object. The induction hypotheses provide evidence for the premises; composing with the corresponding rule morphism gives evidence for the conclusion. Admissible delayed elimination is handled separately in Proposition D.2. □

Proposition D.2 (Soundness of admissible delayed elimination)**.** *Let s be a site equipped with closure support $\kappa_s \Vdash K_s[-]$. For every $n > 0$, the admissible delayed-elimination principle of Proposition 9.1 is semantically sound in every structured proof-relevant model of $\mathcal{H}(A)$.*

Proof. The proof follows the same induction on nested possibility evidence as the syntactic admissibility proof. A proof-relevant element of $\Diamond^n M$ is a chain of n rewrite witnesses ending in evidence for M. The closure-support constructor κ_s maps each witness in that chain to a witness through the displayed context $K_s[-]$. After the transported chain reaches a process of contextual modal type, ordinary modal elimination supplies the final site-interaction witness. The result is a chain of $n+1$ witnesses ending in evidence for $C[\vec{b}/\vec{y}]$, which is exactly evidence for $\Diamond^{n+1}(C[\vec{b}/\vec{y}])$. In proof-irrelevant regular semantics this becomes the corresponding composite of images. □

The preceding theorem is the soundness result needed for the free completion. It does not require the semanticist to define $\Diamond C$ as a least proof-irrelevant may-predicate, nor to define contextual modalities by quantifying over all rely arguments after evidence has been forgotten. Those exact proof-irrelevant interpretations are additional semantic choices.

Proposition D.3 (Proof-irrelevant may is extra structure)**.** *In a regular semantic category, the proof-irrelevant interpretation of the one-step possibility predicate is obtained from the proof-relevant evidence display by taking a regular image. For contextual modalities with discharged rely variables, an exact proof-irrelevant interpretation additionally requires the corresponding dependent universal operation along the rely-variable projection.*

Proof. For an ordinary one-step possibility type, the proof-relevant evidence object in context Γ is

$$\{(P, Q, r, e_C) \mid r : P \rightsquigarrow Q \text{ and } e_C : Q :: C\}.$$

Forgetting Q, r, and e_C gives a map to the object of processes P. In a regular category its image is the proof-irrelevant predicate of processes for which such evidence exists:

$$\mathrm{im}\left(\{(P, Q, r, e_C) \mid r : P \rightsquigarrow Q \text{ and } e_C : Q :: C\} \longrightarrow \mathrm{Pr}\right).$$

This proves the claim for $\Diamond$.

For a contextual modality with rely variables, the proof-irrelevant reading in **Set**, at fixed index data $\vec{x} \in [\![\vec{A}]\!]$, has the form

$$p \in [\![\langle K_s \rangle^{\vec{x}::\vec{A}}_{\vec{y}::\vec{B}} C]\!] \quad \Longleftrightarrow \quad \forall \vec{y} \in [\![\vec{B}(\vec{x})]\!].\exists Q, r, e_C.r : K_s[p][\vec{y}] \rightsquigarrow Q \text{ and } e_C : Q :: C(\vec{x}, \vec{y}).$$

The existential part is an image of rewrite-and-typing evidence. The discharged rely variables contribute the universal quantifier; categorically this is a right adjoint to pullback along the rely-variable projection, or equivalent dependent universal structure. Thus the proof-relevant generated presentation uses explicit evidence and its named rule morphisms, while exact proof-irrelevant may semantics requires the additional logical structure just described. □

References

[1] J. Adámek, J. Rosický, and E. M. Vitale. *Algebraic Theories: A Categorical Introduction to General Algebra*. Cambridge University Press, 2011.

[2] H. Barendregt. Introduction to generalized type systems. *Journal of Functional Programming*, 1(2):125–154, 1991.

[3] H. Barendregt. Lambda calculi with types. In S. Abramsky, D. M. Gabbay, and T. S. E. Maibaum, editors, *Handbook of Logic in Computer Science*, volume 2, pages 117–309. Oxford University Press, 1992.

[4] G. Boudol, *Asynchrony and the Pi-calculus*, Research Report RR-1702, INRIA, 1992.

[5] J. Cartmell. Generalised algebraic theories and contextual categories. *Annals of Pure and Applied Logic*, 32:209–243, 1986.

[6] M. Fiore, G. Plotkin, and D. Turi. Abstract syntax and variable binding. In *Proceedings of the 14th Annual IEEE Symposium on Logic in Computer Science*, pages 193–202, 1999.

[7] K. Honda and M. Tokoro, "An object calculus for asynchronous communication", in *ECOOP '91: European Conference on Object-Oriented Programming*, Lecture Notes in Computer Science, vol. 512, Springer, 1991, pp. 133–147.

[8] B. Jacobs. *Categorical Logic and Type Theory.* North-Holland, 1999.

[9] J. Lambek and P. J. Scott. *Introduction to Higher-Order Categorical Logic.* Cambridge University Press, 1986.

[10] F. W. Lawvere, *Functorial Semantics of Algebraic Theories*, Ph.D. thesis, Columbia University, 1963.

[11] J. J. Leifer and R. Milner. Deriving bisimulation congruences for reactive systems. In *CONCUR 2000*, Lecture Notes in Computer Science 1877, pages 243–258. Springer, 2000.

[12] R. Milner, J. Parrow, and D. Walker, "A calculus of mobile processes, I", *Information and Computation* **100** (1992), no. 1, 1–40.

[13] A. N. Whitehead, *A Treatise on Universal Algebra, with Applications. Vol. I*, Cambridge University Press, Cambridge, 1898.